

UNIT - II

2

Regular Expressions

Syllabus

*Introduction, Operators of RE, Precedence of operators, Algebraic laws for RE, Language to Regular Expressions, Equivalence of two REs. **Conversions:** RE to NFA, DFA, DFA to RE using Arden's theorem, Pumping Lemma for Regular languages, Closure and Decision properties of Regular languages. Myhill-Nerode theorem.*

Contents

2.1 Operators of RE	Dec.-16, Oct.-19,	Marks 6
2.2 Language to Regular Expressions	Dec.-07, 11, 12, 13, 14, 15,	
	May-08, 09, 10, 11, 13, 14, 16,	
	Aug.-15, Oct.-16, 17, 18, 19,	Marks 8
2.3 Precedence of Operators		
2.4 Algebraic Laws for RE	Aug.-17,	Marks 3
2.5 Equivalence of Two Regular Expressions	May-06, 07, 09, 11, Dec.-08, 09, 11,	
	Oct.-16, Aug.-17,	Marks 10
2.6 RE to DFA	May-10, 12, 14, 15, 19, Dec.-11, 13, 14,	
	16, 18, 19, Aug.-15, 17, Oct.-18, 19,	
		Marks 10
2.7 DFA to RE using Arden's Theorem	May-09, 10, 14, 15, 16, Dec.-08, 09, 10,	
	13, 14, 15, Aug.-15,	Marks 10
2.8 Pumping Lemma for Regular Languages	Dec.-06, 08, 12, May-06, 07, 09, 12, 13,	
	14, 16, Aug.-17, Oct.-18,	Marks 8
2.9 Closure Properties of Regular Languages	May-18, Oct 18,	Marks 6
2.10 Decision Properties of Regular Languages		
2.11 Limitation of FA	Aug.-15,	Marks 2
2.12 Myhill - Nerode Theorem		
2.13 Case Study	Dec.-05, 06, 10, 13, 14, May-13, 14,	
	Oct.-19,	Marks 6
2.14 Fill in the Blanks		
2.15 Multiple Choice Questions		

2.1 Operators of RE

SPPU : Dec.-16, Oct.-19, Marks 6

Let Σ be an alphabet which is used to denote the input set. The regular expression over Σ can be defined as follows.

Definition

1. ϕ is a regular expression which denotes the **empty set**.
2. ϵ is a regular expression and denotes the set $\{\epsilon\}$ and it is a null string.
3. For each ' a ' in Σ ' a ' is a regular expression and denotes the set $\{a\}$.
4. If r and s are regular expressions denoting the languages L_1 and L_2 respectively, then
 - $r + s$ is equivalent to $L_1 \cup L_2$ i.e. union.
 - rs is equivalent to $L_1 L_2$ i.e. concatenation.
 - r^* is equivalent to L_1^* i.e. closure.

The r^* is known as **kleen closure** or closure which indicates occurrence of r for ∞ number of times.

For example if $\Sigma = \{a\}$ and we have regular expression $R = a^*$, then R is a set denoted by $R = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

That is R includes any number of a 's as well as empty string which indicates zero number of a 's appearing, denoted by ϵ character.

Similarly there is a **positive closure** of L which can be shown as L^+ . The L^+ denotes set of all the strings except the ϵ or null string. The null string can be denoted by ϵ or $\wedge$

If $\Sigma = \{a\}$ and if we have regular expression $R = a^+$ then R is a set denoted by

$$R = \{a, aa, aaa, aaaa, \dots\}$$

We can construct L^* as

$$L^* = \epsilon \cdot L^+$$

Example 2.1.1 What is kleen closure ? What is positive closure ? For a given language L under what circumstances will L^+ and L^* be equal ? **SPPU : Dec.-16, End Sem, Marks 6**

Solution :

- The Kleen closure is a set of strings of any length (including null string ϵ).
- The positive closure Σ^+ can be defined as $\Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots$. That means positive closure consists of all strings of any length except null string.
- Basically $L^+ \subseteq L^*$ But $L^+ = L^*$ if and only if $\epsilon \in L^+$. The $\epsilon \in L^+$ if and only if $\epsilon \leftarrow L$. That mean ϵ is the shortest word in L .

Review Question

1. Define the following terms : i) Kleen closure ii) Positive closure.

SPPU : Oct.-19, In Sem, Marks 2

2.2 Language to Regular Expressions

SPPU : Dec.-07, 11, 12, 13, 14, 15, May-08, 09, 10, 11, 13, 14, 16, Aug.-15, Oct.-16, 17, 18, 19, Marks 8

Let us try to use regular expressions with the help of some examples.

Example 2.2.1 Write the regular expression for the language accepting all combinations of a 's over the set $\Sigma = \{a\}$.

Solution : All combinations of a 's means a may be single, double, tripple and so on. There may be the case that a is appearing for zero times, which means a null string. That is we expect the set of $\{\epsilon, a, aa, aaa, \dots\}$. So we can give regular expression for this as

$$R = a^*$$

That is kleen closure of a .

Example 2.2.2 Design the regular expression (r.e.) for the language accepting all combinations of a 's except the null string over $\Sigma = \{a\}$.

Solution : The regular expression has to be built for the language

$$L = \{a, aa, aaa, \dots\}$$

This set indicates that there is no null string. So we can denote r.e. as

$$R = a^+$$

As we know, positive closure indicates the set of strings without a null string.

Example 2.2.3 Design regular expression for the language containing all the strings containing any number of a 's and b 's.

Solution : The regular expression will be

$$\text{r.e.} = (a + b)^*$$

This will give the set as $L = \{\epsilon, a, aa, ab, b, ba, bab, abab, \dots\}$ any combination of a and b .

The $(a + b)^*$ means any combination with a and b even a null string.

Example 2.2.4 Construct the regular expression for the language containing all strings having any number of a 's and b 's, except the null string.

Solution : $\text{r.e.} = (a + b)^+$

This regular expression will give the set of strings of any combination of a 's and b 's except a null string.

Example 2.2.5 Construct the r.e. for the language accepting all the strings which are ending with 00 over the set $\Sigma = \{0, 1\}$.

Solution : The r.e. has to be formed in which at the end, there should be 00. That means

$$\text{r.e.} = (\text{any combination of 0's and 1's}) 00$$

$$\text{i.e.} \quad \text{r.e.} = (0+1)^* 00$$

Thus the valid string are 100, 0100, 1000, 10100 strings ending with 00.

Example 2.2.6 Write r.e. for the language accepting the strings which are starting with 1 and ending with 0, over the set $\Sigma = \{0, 1\}$.

Solution : The first symbol in r.e. should be 1 and the last symbol should be 0.

$$\text{So,} \quad R = 1(0+1)^* 0$$

Note that the condition is strictly followed by keeping starting and ending symbols correctly. In between them there can be any combination of 0 and 1 including a null string.

Example 2.2.7 If $L = \{\text{The language starting and ending with } a \text{ and having any combination of } b\text{'s in between}\}$, then what is r ?

Solution : The regular expression

$$r = a b^* a$$

Example 2.2.8 Describe in simple English the language represented by the following regular expression $r = (a + ab)^*$

Solution : We will first try to find out the set of strings, which can be possible by this r ,

$$L(r) = \{a, aba, abab, aab, aaa, \dots\}$$

The language is beginning with a but not having consecutive (in a clump) b 's.

Example 2.2.9 Write regular expression to denote the language L over Σ^* , where $\Sigma = \{a, b, c\}$ in which every string will be such that any number of a 's is followed by any number of b 's is followed by any number of c 's.

Solution : As we have seen any number of a 's means a^* any number of b 's means b^* any number of c 's means c^* . Since as given in problem statement, b 's appear after a 's and c 's appear after b 's. So the regular expression could be -

$$r = a^* b^* c$$

Example 2.2.10 Write a regular expression to denote a language L over Σ^* , where $\Sigma = \{a, b, c\}$ such that every string will have atleast one a followed by atleast one b followed by atleast one c .

Solution : Now, in this problem the condition is slightly changed to "atleast". That means the null string is not allowed at all. So, we can write

$$R = a^+ b^+ c^+$$

Example 2.2.11 Write r.e. to denote a language L which accepts all the strings which begin or end with either 00 or 11.

Solution : The r.e. can be categorized into two subparts.

$$R = L_1 + L_2$$

L_1 = The strings which begin with 00 or 11.

L_2 = The strings which end with 00 or 11.

Let us find out L_1 and L_2 .

L_1 = (00 + 11) (any number of 0's and 1's)

L_1 = (00 + 11) (0+1)*

Similarly,

L_2 = (any number of 0's and 1's) (00 + 11)

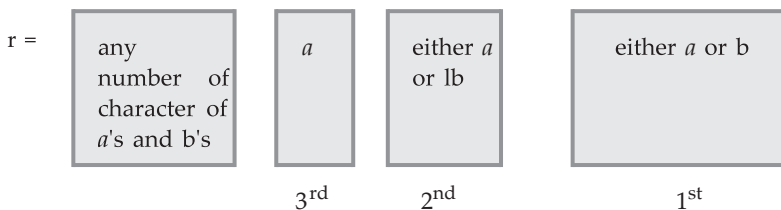
= (0 + 1)* (00 + 11)

Hence

$$R = [(00+11)(0+1)^*] + [(0+1)^*(00+11)]$$

Example 2.2.12 Write a r.e. to denote a language L over Σ^* , where $\Sigma = \{a, b\}$ such that the 3rd character from right end of the string is always a .

Solution : The r.e. contains the third symbol from right end as a



$$r = (a+b)^* a (a+b) (a+b)$$

Thus the valid strings are babb or baaa or baba or aaab and so on.

Example 2.2.13 Construct r.e. for the language L which accepts all the strings with atleast two b 's over the $\Sigma = \{a, b\}$.

Solution : $R = (a + b)^* b (a + b)^* b (a + b)^*$

Atleast two b 's are maintained but the two b 's are surrounded by any combination of a 's and b 's.

Example 2.2.14 Construct r.e. for the language which consists of exactly two b 's over the set $\Sigma = \{a, b\}$.

Solution : Now, this problem is similar to the previous problem. But only difference is that there should be exactly two b's.

$$R = a^* b a^* b a^*$$

a^* indicates either the string contains any number of a 's or a null string. Thus we can derive any string having exactly two b's and any number of a 's.

Example 2.2.15 Write r.e. which contains L having strings which should have atleast one 0 and atleast one 1.

Solution : The required expression will be

$$R = [(0+1)^* 0 (0+1)^* 1 (0+1)^*] + [(0+1)^* 1 (0+1)^* 0 (0+1)^*]$$

Example 2.2.16 Construct r.e. which denotes a language L over the set $\Sigma = \{0\}$ having even length of string.

Solution : Since $\Sigma = \{0\}$ there are strings in L of even length i.e.

$$L = \{\epsilon, 00, 0000, 000000, \dots\}.$$

So we can give the regular expression as

$$R = (00)^*$$

Thus kleen closure indicates the recursion of two zeros which will ultimately give the even length of the string.

Example 2.2.17 Write r.e. which denotes a language L over the set $\Sigma = \{1\}$ having odd length of strings.

Solution : $R = 1 (11)^*$

The odd length strings are $L = \{1, 111, 11111, \dots\}$. Note the difference between previous example and this one.

Example 2.2.18 Describe the language denoted by following regular expression
r.e. = $(b^*(aaa)^*b^*)^*$

Solution : The language can be predicted from the r.e. by finding out the meaning of it. We can split the r.e. as

$$\text{r.e.} = (\text{any combination of b's}) (aaa)^* (\text{any combination of b's}).$$

$L = \{\text{The language consists of the strings in which } a\text{'s appear trippled, there is no restriction on number of b's}\}$

Example 2.2.19 Construct regular expression for the language L over the set $\Sigma = \{a, b\}$ in which the total number of a 's are divisible by 3.

Solution : The total number of a 's are divisible by 3. So the valid strings can be

$$L = \{baaa, bababa, ababab, \dots\}$$

The regular expression can be

$$\text{r.e.} = (b^* a b^* a b^* a b^*)^*$$

Example 2.2.20 Write the r.e. to denote the language L over $\Sigma = \{a, b\}$ such that all the strings do not contain the substring "ab".

Solution : The $L = \{\epsilon, a, b, bb, aa, ba, baa \dots\}$

The $\text{r.e.} = (b^* a^*)$

In this regular expression if we substitute $a^* = \epsilon$ we get all combination of b 's and similarly if we substitute $b^* = \epsilon$ then we will get all combinations of a 's. We have strictly maintained a condition for not to have ab as substring by giving the regular expression as $b^* a^*$.

Example 2.2.21 Find all possible regular expression $L \subseteq \{a, b\}^*$ for

- The set of all strings ending in b
- The set of all strings ending in ba
- The set of all strings ending neither b nor ba .

SPPU : Dec.-07, Marks 6

Solution :

- a) The regular expression for language containing strings ending with b is

$$\text{r.e.} = (a + b)^* b.$$

- b) The regular expression for language containing strings ending with ba is

$$\text{r.e.} = (a + b)^* ba$$

- c) The regular expression for language containing strings ending with neither b nor a is

$$\text{r.e.} = (a + b)^* a^+ b^*$$

Example 2.2.22 Let L be a language. It is clear from the definition that $L^+ \subseteq L^*$. Under what circumstances are they equal ?

SPPU : Dec.-07, May-08, Marks 4

Solution : The language L^+ denotes any combination of input symbols without ϵ .

The language L^* denotes any combination of input symbols along with ϵ .

$$\text{Hence } L^+ \subseteq L^*$$

But if $L_1 = \epsilon + L^+$ then L_1 becomes equal to L^* . That means if ϵ is added as one symbol in the set generated by L^+ then L^* become equal.

Example 2.2.23 Find all possible regular expression $L \subseteq \{a, b\}^*$ for

- The set of all strings ending in ab
- The set of all strings ending in ba
- The set of all strings ending neither ab nor ba .

SPPU : May-08, Marks 6

Solution :

- a) The regular expression is

$$\text{r.e.} = (a + b)^* ab$$

- b) The regular expression is

$$\text{r.e.} = (a + b)^* ba$$

- c) The regular expression is

$$(a + b)^* aa + (a + b)^* bb$$

Example 2.2.24 Consider the two regular expressions $r = 0^* + 1^*$ $s = 01^* + 10^* + 1^*0 + (0^*1)^*$.

- Find the string corresponding to r but not to s
- Find the string corresponding to s but not to r
- Find the string corresponding to both r and f .

SPPU : May-10, Marks 8

Solution :

- The string corresponding to r but not to s : $\{00,000,0000, \dots\}$
- The string corresponding to s but not to r : $\{01 \text{ or } 10\}$
- The string corresponding to both r and f : $\{0, 1, 11, 111, 1111, 11111, \dots\}$

Example 2.2.25 Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$

- The language of all strings containing exactly two 0's.
- The language of all strings containing at least two 0's.
- The expression of all strings that do not end with 01.

SPPU : May-09, Marks 6

Solution :

- If a language contains all strings containing exactly two 0's then, it can be at any position, may or may not be surrounded by 1's. Hence r.e. Will be

$$\text{r.e.} = 1^* 0 1^* 0 1^*$$
- If a language contains all strings containing at least two 0's, then it may be at any position, and may or may not be surrounded by $(0+1)^*$. Hence r.e. will be

$$\text{r.e.} = (0+1)^* 0 (0+1)^* 0 (0+1)^*$$
- The language contains all the strings that do not end with 01 means it may end with either 0's or end with either 1's or end with 10. Hence r.e. will be -

$$\text{r.e.} = [(0+1)^* 0]^* + [(0+1)^* (10)^*]$$

Example 2.2.26 Obtain the regular expressions for the following sets :

- 1) $L1 = \{b^2, b^5, b^8, b^{11}, b^{14} \dots\}$ 2) $L2 = \{a^{2n+1} | n > 0\}$

SPPU : May-11, Marks 4

Solution :

1) $L1 = \{b^2, b^5, b^8, b^{11}, b^{14} \dots\}$

The regular expression for $L1 = \{b^2, b^5, b^8, b^{11}, \dots\}$ is

Regular expression = $bb(bbb)^*$

2) $L2 = \{a^{2n+1} | n > 0\}$

The regular expression for $L2 = \{a^{2n+1} | n > 0\}$ is

Regular expression = $a(aa)^+$

Example 2.2.27 Obtain in plain english language represented by following regular expression.

- a) $0^*(10^*10^*)^*1(0^*10^*1)^*0^*$ b) $0^*(0^*10^*1)^*0^*$

SPPU : May-11, Marks 4

Solution :

a) $0^*(10^*10^*)^*1(0^*10^*1)^*0^*$:

$L = \{\text{The language containing group of even number of 1's separated by single 1}\}$

b) $0^*(0^*10^*1)^*0^*$:

$L = \{\text{The language containing even number of 1's and any number of 0's}\}$

Example 2.2.28 Represent following formal languages using regular expressions.

- 1) All string's of a's and b's without any combination of double letters.
 2) All string's of 0's and 1's with even number of 0's. 3) All string's of a's and b's containing at least two a's.

SPPU : May-11, Marks 6

Solution :

1) All string's of a's and b's without any combination of double letters : $(01)^* + (10)^*$

2) All string's of 0's and 1's with even number of 0's : $(1^*01^*01^*)^*$

3) All string's of a's and b's containing at least two a's : $(b^*ab^*ab^*)^+$

Example 2.2.29 Find the regular expressions representing the following sets :

- i) The set of all strings over $\{a, b\}$ with three consecutive b's.
 ii) The set of all strings over $\{0, 1\}$ beginning with 00.
 iii) The set of all strings over $\{0, 1\}$ ending with 00 and beginning with 1.

SPPU : Dec.-11, Marks 6

Solution :

i) $\text{r.e.} = a^*(bbb)^*a^*$

ii) $\text{r.e.} = 00(0+1)^*$

iii) $\text{r.e.} = 1(0+1)^*00$

Example 2.2.30 Find regular expressions representing the following sets :

- i) The set of all strings over $\{a, b\}$ having atmost one pair of a's or atmost one pair of b's.
- ii) The set of all strings over $\{a, b\}$ in which the number of occurrences of a is divisible by 3.
- iii) The set of all strings over $\{a, b\}$ in which there are at least two occurrences of b between any two occurrences of a.

SPPU : Dec.-12, Marks 6

Solution :

- i) The set of all strings over $\{a, b\}$ having atmost one pair of a's or atmost one pair of b's.

$$((ab)+b)^*a a ((ba)+b)^*/((ba)+a)^*bb((ab)+a)^*$$

- ii) The set of all strings over $\{a, b\}$ in which the number of occurrence of a is divisible by 3.

$$\Rightarrow b^*(aaa)^*b^*$$

- iii) The set of all strings over $\{a, b\}$, in which there are at least two occurrences of b between any two occurrences of a.

$$\Rightarrow (a+b)^*(aa)(bb)^+(aa)(a+b)^*$$

Example 2.2.31 Write R.E. for the following

- i) $\Sigma = (0, 1)$ odd number of 1's in strings
- ii) $\Sigma = (0, 1)$ triple 0 must never appear in string.

SPPU : May-14, Marks 6

Solution :

i) $\text{r.e.} = (0^*10^*)^*(10^*10^*)^*$

ii) $\text{r.e.} = (1+01)^*00(1+10)^*$

Example 2.2.32 Give english description of the language of the following regular expression

$$(1 + \epsilon)(00^*1)^*0^*$$

SPPU : Dec.-13, Marks 6

Solution : This expression generates the strings as -

$$\{ \epsilon, 1, 101, 1001, 10, 100, 1010, \dots \}$$

This regular expression generates all strings in which every 1 is separated by one or more 0. It also accepts the string which has only one 1 or the empty string.

Example 2.2.33 Write regular expressions for the following languages over $\{0, 1\}^*$

- i) The set of strings that begin with 110
- ii) The set of all strings not containing 101 as a substring.

SPPU : Dec.-13, Marks 6

Solution :

- i) r.e. = $110(0 + 1)^*$
- ii) r.e. = $(0 + 11^*00)^*(\epsilon + 11^*(\epsilon + 0))$

Example 2.2.34 Write regular expressions for each of the following languages :

- i) For $\Sigma = \{a, b\}$, set of all strings with no consecutive 'a's and 'b's.
- ii) For $\Sigma = \{0, 1\}$, set of all strings in which every 0 is immediately followed by at least two 1's.
- iii) $L = \{1^{(2n+1)} \mid n \geq 1\}$

SPPU : Dec.-14, In Sem, Marks 6

Solution : i) $[(ab)^* + (ba)^*]$ ii) $[(011 + 1)^*]$ iii) $[1(11)^*]$

Example 2.2.35 Describe the languages accepted by the following Regular Expressions and justify with suitable examples :

- i) $a^* \cdot (a + b)^* \cdot ab$
- ii) $(1^* . 01^* . 01^*)^*$
- iii) $a^* b + b^* a$

SPPU : Aug.-15, In Sem, Marks 6

Solution :

- i) The language containing the strings that begins with single a and end with 'ab'.
- ii) The language containing the strings having even number of 0's.
- iii) The language containing strings that either begin with any number of a's and end with single b or begin with any number of b's and end with single a.

Example 2.2.36 Write regular expressions for the following languages over the alphabet

$$\Sigma = \{a, b\}$$

- i) All strings that do not end with 'aa'.
- ii) All strings that contain an even number of 'b's.
- iii) All strings which do not contain the substring 'ba'.

SPPU : Dec.-15, End Sem, Marks 6

Solution :

- i) The language contains all the strings that do not end with aa means it may end with bb, ba or ab. Hence r. e. will be
 $(a + b)^* ((ab)^* + (ba)^* + b^*)$
- ii) r. e. = $(a^* b a^* b)^*$
- iii) The $L = \{\epsilon, a, b, bb, aa, ab, abb, \dots\}$
r. e. = $(a^* b^*)$

Example 2.2.37 Determine a regular expression over the alphabet $\{x, y\}$ For the following :

- i) All strings containing exactly two x 's.
- ii) All strings that do not end with xy .
- iii) All strings starting with yy .

SPPU : May-16, End Sem, Marks 6

Solution :

- i) The regular expression is

$$\text{r.e} = y^* xy^* xy^*$$

- ii) The strings do not end with xy means string can end with x or y or with yx .

The r.e. will then be –

$$(x + y)^* yx + (x + y)^* x + y^*$$

- iii) The regular expression will be

$$yy(x + y)^*$$

Example 2.2.38 Give the regular expression for the following languages :

- i) $L = \{x \mid x \in \{a, b\}^* \text{ and } x \text{ contains exactly two } a\text{'s}\}$
- ii) $L = \{a, c, ab, cb, abb, cbb, abbb, cbbs, \dots\}$
- iii) $L = \{x \mid x \in \{a, b\}^* \text{ and } x \text{ is any string that begins in "abb" or } a\}$

SPPU : Oct.-16, In Sem, Marks 6

Solution :

- i) $b^* ab^* ab^*$

- ii) The set contains all the string that either begin with a or c but always end with any number of b 's.

$$\therefore \text{r.e.} = (a + c)b^*$$

- iii) $\text{r.e.} = (abb + a)(a + b)^*$

Example 2.2.39 Give the regular expression for the following languages.

- i) The set of strings over the alphabet $\{a, b\}$ starting with b and ending with odd number of a 's or even number of b 's.
- ii) The set $\{10, 1010\}$

SPPU : Oct.-18, In Sem, Marks 3

Solution : i) $\text{r.e.} = b(a(aa)^* + (bb)^*)$ ii) $\text{r.e.} = (10)^+$

Example 2.2.40 Determine a regular expression over the alphabet $\Sigma = \{a, b\}$.

- i) All strings that contain an even number of 'b's
- ii) All strings that do not end with 'aa'

SPPU : Oct.-19, In Sem, Marks 4

Solution :

i) $\text{r.e.} = (a^* ba^* ba^*)^*$

ii) $\text{r.e.} = (a + b)^* (ab + ba + bb)$

Example 2.2.41 Justify if true or false the following :

Every subset of a regular language is regular

SPPU : Oct.-19, In Sem, Marks 3

Solution : Every subset of regular language is regular is **False**

Justification :

Let, $a^* b^*$ be a regular expression. The subset of regular expression $a^* b^*$ is $a^n b^n$.

For $a^* b^*$ we can draw DFA but for $a^n b^n$ we can not draw DFA.

Hence, $a^n b^n$ is said to be non-regular expression.

This shows that subset of regular expression is non-regular expression.

Review Questions

1. Define with example - Regular expression.

SPPU : May-13, Marks 2

2. Define with suitable example - Regular expression and Operation.

SPPU : Aug.-17, In Sem, Marks 3

2.3 Precedence of Operators

There are three main operators that can be used in RE - **Union**, **concatenation** and **kleen closure**. For regular expressions, the following is the order of precedence for the operators :

- i) The **star operator (kleen closure)** is having highest precedence. That means the r.e. with star operator must be grouped first.
- ii) The next precedence is for **dot operator** i.e. for concatenation. The concatenation is an **associative** operator, hence the way we group the elements of regular expression does not matter much. For example 0110 can be grouped as (01)(10), or 0(110) or (011)0
- iii) Final precedence is given to + (**i.e. Union**) **operation**. The union or + operator is also associative operator.

2.4 Algebraic Laws for RE

SPPU : Aug.-17, Marks 3

The two regular expressions P and Q are equivalent (denoted as $P = Q$) if and only if P represents the same set of strings as Q does. For showing this equivalence of regular expressions we need to show some identities of regular expressions.

Let P, Q and R are regular expressions then the identity rules are as given below.

1. $\epsilon R = R\epsilon = R$
2. $\epsilon^* = \epsilon$ ϵ is null string
3. $(\phi)^* = \epsilon$ ϕ is empty string.
4. $\phi R = R\phi = \phi$
5. $\phi + R = R$
6. $R + R = R$
7. $RR^* = R^*R = R^+$
8. $(R^*)^* = R^*$
9. $\epsilon + RR^* = R^*$
10. $(P + Q)R = PR + QR$
11. $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$
12. $R^*(\epsilon + R) = (\epsilon + R)R^* = R^*$
13. $(R + \epsilon)^* = R^*$
14. $\epsilon + R^* = R^*$
15. $(PQ)^*P = P(QP)^*$
16. $R^*R + R = R^*R$

Review Question

1. What are algebraic laws of regular expression.

SPPU : Aug.-17, In Sem, Marks 3

2.5 Equivalence of Two Regular Expressions

SPPU : May-06, 07, 09, 11, Dec.-08, 09, 11, Oct.-16, Aug.-17, Marks 10

Let us see one important theorem named **Arden's Theorem** which helps in checking the equivalence of two regular expressions.

Arden's Theorem : Let, P and Q be the two regular expressions over the input set Σ . The regular expression R is given as

$$R = Q + RP$$

Which has a unique solution as $R = QP^*$.

Proof : Let, P and Q are two regular expressions over the input string Σ .

If P does not contain ϵ then there exists R such that

$$R = Q + RP \quad \dots (2.5.1)$$

We will replace R by QP^* in equation (2.5.1)

Consider R.H.S. of equation (2.5.1)

$$\begin{aligned} &= Q + QP^*P \\ &= Q(\epsilon + P^*P) \\ &= QP^* \end{aligned} \quad \because \epsilon + R^*R = R^*$$

Thus

$$R = QP^*$$

is proved. To prove that $R = QP^*$ is a unique solution, we will now replace L.H.S. of equation 1 by $Q + RP$. Then it becomes

$$Q + RP$$

But again R can be replaced by $Q + RP$.

$$\begin{aligned} \therefore Q + RP &= Q + (Q + RP)P \\ &= Q + QP + RP^2 \end{aligned}$$

Again replace R by $Q + RP$.

$$\begin{aligned} &= Q + QP + (Q + RP)P^2 \\ &= Q + QP + QP^2 + RP^3 \end{aligned}$$

Thus if we go on replacing R by $Q + RP$ then we get,

$$\begin{aligned} Q + RP &= Q + QP + QP^2 + \dots + QP^i + RP^{i+1} \\ &= Q(\epsilon + P + P^2 + \dots + P^i) + RP^{i+1} \end{aligned}$$

From equation (2.5.1),

$$R = Q(\epsilon + P + P^2 + \dots + P^i) + RP^{i+1} \quad \dots (2.5.2)$$

Where $i \geq 0$

Consider equation (2.5.2),

$$\begin{aligned} R &= Q \underbrace{(\epsilon + P + P^2 + \dots + P^i)}_{P^*} + RP^{i+1} \\ \therefore R &= QP^* + RP^{i+1} \end{aligned}$$

Let w be a string of length i.

In RP^{i+1} has no string of less than $i + 1$ length. Hence w is not in set RP^{i+1} . Hence R and QP^* represent the same set. Hence it is proved that

$R = Q + RP$ has a unique solution.

$$R = QP^*.$$

Problems for Equivalence of Two

Example 2.5.1 Prove $(1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)=0^*1(0+10^*1)^*$

SPPU : Dec.-11, Marks 4

Solution : Let us solve L.H.S. first,

$$(1+00^*1)+(1+00^*1)(0+10^*1)^*(0+10^*1)$$

We will take $(1+00^*1)$ as a common factor

$$1+00^*1 \quad \underbrace{(\epsilon + (0+10^*1)^*(0+10^*1))}_{\downarrow}$$

$$(\epsilon + R^*R) \quad \text{where } R = (0+10^*1)$$

As we know, $(\epsilon + R^*R) = (\epsilon + RR^*) = R^*$

$\therefore (1+00^*1)((0+10^*1)^*)$ out of this consider

$$\underbrace{(1+00^*1)}_{\downarrow} (0+10^*1)^*$$

Taking 1 as a common factor

$$(\epsilon + 00^*) 1 (0+10^*1)^*$$

Applying $\epsilon + 00^* = 0^*$

$$0^*1(0+10^*1)^* = \text{R.H.S.}$$

Hence the two regular expressions are equivalent.

Example 2.5.2 Show that $(0^*1^*)^* = (0+1)^*$.

Solution :

Consider L.H.S. $= (0^*1^*)^*$

$$= \{\epsilon, 0, 00, 1, 11, 111, 01, 10, \dots\}$$

= {any combination of 0's, any combination of 1's, any
combination of 0 and 1, ϵ }

Similarly,

$$\text{R.H.S.} = (0+1)^*$$

$$= \{\epsilon, 0, 00, 1, 11, 111, 01, 10, \dots\}$$

$$= \{\epsilon, \text{any combination of 0's, any combination of 1's,} \\ \text{any combination of 0 and 1}\}$$

Hence, L.H.S. = R.H.S. is proved.

Example 2.5.3 Show that $(ab)^* \neq (a^*b^*)$

SPPU : Oct.-16, In Sem, Marks 2

Solution : Consider

$$\text{L.H.S.} = (ab)^* = \{\epsilon, ab, abab, ababab, \dots\}$$

$$\text{and R.H.S.} = (a^*b^*) = \{\epsilon, a, aa, aa, b, bb, bbb, ab \dots\}$$

Note that in the L.H.S, the r.e. will give the strings of 'ab' combination i.e. ab, abab, ababab and so on. But in R.H.S. there is possibility of ab but not of abab and so on.

Thus L.H.S. $\neq$ R.H.S. is proved.

Example 2.5.4 Show that $(r+s)^* \neq r^*s^*$

Solution : Let us consider the L.H.S. first,

$$\begin{aligned} \text{L.H.S.} &= (r+s)^* \\ &= \{\epsilon, r, rr, s, ss, rs, sr, rsrs, \dots\} \\ &= \{\epsilon, \text{any combination of } r \text{ and } s\} \end{aligned}$$

$$\text{R.H.S.} = \{\epsilon, r, rr, rrr, s, ss, \dots\}$$

$\{\epsilon, \text{any combination of only } r \text{ or any combination of only } s\}$

Note that in R.H.S. there is no combination of r and s together. Hence

L.H.S. $\neq$ R.H.S. is proved.

Example 2.5.5 Prove that $r(s+t) = rs+rt$

Solution : In case of L.H.S.

$$\begin{aligned} \text{L.H.S.} &= r(s+t) \quad \text{solving this further} \\ &= r \cdot s + r \cdot t \end{aligned}$$

i.e. r is concatenated with s or with t

$$= \text{R.H.S.}$$

L.H.S. = R.H.S. is proved.

Example 2.5.6 Check for the following regular expressions for equivalence and justify.

$$R_1 = [(a+bb)^*(b+aa)^*]^*$$

$$R_2 = (a+b)^*$$

Solution : Here R_1 and R_2 are equivalent because the strings which can be generated by R_1 can be generated by R_2 or vice versa.

For instance, consider,

$R_1 = abab$ – this string can be generated by selecting **a** from $(a + bb)^*$ then **b** from $(b + aa)^*$ again **a** from $(a + bb)^*$ and **b** from $(b + aa)^*$.

$R_2 = abab$ – this string can be selecting **a** then **b** repetitively from $(a + b)^*$

Thus $R_1 = R_2$ is proved

Example 2.5.7 $R_1 = (a + b)^* abab^*$
 $R_2 = b^* a (a + b)^* ab^*$

Solution : Here strings generated by R_1 are same as R_2

For instance R_1 can generate **abababb**. Similarly R_2 can generate abababb.

Thus $R_1 = R_2$ is proved.

Example 2.5.8 Prove $\varepsilon + 1^* (011)^* (1^* (011)^*)^* = (1 + 011)^*$

Solution : Let, L.H.S. is

$$\varepsilon + \underbrace{1^* (011)^*}_{P_1} \underbrace{(1^* (011)^*)^*}_{P_1^*}$$

If we consider $1^* (011)^*$ as P_1 then,

$$= \varepsilon + P_1 P_1^*$$

$$= P_1^*$$

$$\because \varepsilon + P_1 P_1^* = P_1^*$$

We can put $P_1 = 1^* (011)^*$ then,

$$= (1^* (011)^*)^*$$

Now consider $P_2 = 1$ and $P_3 = (011)$ then it becomes

$$= (P_2^* P_3^*)^*$$

$$= (P_2 + P_3)^*$$

$$\because (P^* Q^*)^* = (P + Q)^*$$

$$= (1 + 011)^*$$

$$= \text{R.H.S.}$$

Thus L.H.S. = R.H.S. Hence

$\varepsilon + 1^* (011)^* (1^* (011)^*)^* = (1 + 011)^*$ is proved.

Example 2.5.9 Prove or disprove the following for regular expression r , s and t .

$$a) (rs+r)^* r = r (sr+r)^* \quad b) s(rs+s)^* r = rr^* s(rr^* s)^*$$

$$c) (r+s)^* = r^* + s^*$$

SPPU : May-07, Marks 10, Aug.-17, In Sem, Marks 3

Solution :

$$\text{a) } (rs + r)^* r = r (sr + r)^*$$

$$\begin{aligned} \text{Let } r_1 &= (rs + r)^* r \\ &= \{\epsilon, rs, r, rsrs, rsr, rsr, rr...\} r \\ &= \{\epsilon, rsr, rr, rsrsr, rsrr, rsrr, rrr \dots\} \\ r_2 &= r (sr + r)^* \\ &= r \{\epsilon, sr, r, sr sr, srr, rsr, rr \dots\} \\ &= \{\epsilon, rsr, rr, sr srr, r \dots\} \end{aligned}$$

Here L.H.S. = R.H.S.

$$\text{Hence } (rs + r)^* r = r (sr + r)^*$$

$$\text{b) } s(rs + s)^* r = rr^* s (rr^* s)^*$$

$$\begin{aligned} \text{Let } r_1 &= s(rs + s)^* r \\ &= s\{\epsilon, rs, s, rss, srs, rsrs, ss \dots\} r \\ &= \{\epsilon, rss, ss, rsss, srss, rsrss, sss \dots\} r \\ &= \{\epsilon, rrss, ssr, rrsss, srssr, rsrssr, sssr\} \end{aligned}$$

$$\begin{aligned} \text{Let } r_2 &= rr^* s (rr^* s)^* \\ &= r \{\epsilon, r, rr, rrr, rrrr \dots\} s \\ &= (r \{\epsilon, r, rr, rrr, rrrr \dots\} s)^* \\ &= rs \{\epsilon, r, rr, rrr, rrrr \dots\} \\ &= \{rs\} \cup \{\epsilon, r, rr, rrr, rrrr \dots\} \\ &= \{\epsilon, rrs, rrrs, rrrrs, rrrrrs \dots\} \\ &= \{\epsilon, rrs, rrrs, rrrrs, rrrrrs\} \\ &= \{\epsilon, rrss, ssr, rrsss \dots\} \end{aligned}$$

Here L.H.S. = R.H.S.

$$\text{Hence } s(rs + s)^* r = rr^* s (rr^* s)^*$$

$$\text{c) } (r + s)^* = r^* + s^*$$

$$\begin{aligned} \text{Let } r_1 &= (r + s)^* \\ &= (\epsilon, r, s, rs, sr, rsr, srs \dots) \end{aligned}$$

$$\begin{aligned} \text{Let } r_2 &= r^* + s^* \\ &= (\epsilon, r, rr, rrr, rrr\dots) \cup (\epsilon, s, ss, sss, ssss) \\ &= (\epsilon, r, s, s, rs, sr, rsr, srs \dots) \end{aligned}$$

Here L.H.S. = R.H.S.

Hence $(r + s)^* = r^* + s^*$

Example 2.5.10 Prove a) $\phi^* = \epsilon$ b) $(r^* s^*)^* = (r + s)^*$

SPPU : Dec.-08, Marks 4, Oct.-16, In Sem, Marks 2

Solution :

a) Let L.H.S. = ϕ^*

$$= \{\epsilon, \phi, \phi\phi, \phi\phi\phi, \dots\}$$

$$\because R^* = \{\epsilon, R, RR, RRR, \dots\}$$

$$= \{\epsilon\} \text{ or empty set}$$

$$\because \phi R = R\phi = \phi \text{ i.e. empty set.}$$

$$= \epsilon$$

$$= \text{R.H.S.}$$

As, L.H.S. = R.H.S. , $\phi^* = \epsilon$ is proved.

b) Let, $(r^* s^*)^*$

$$\text{L.H.S.} = \{\epsilon, r, rr, rs, sr, rrs, srr, srs, \dots\}$$

$$= \epsilon + \text{any combination of } r \text{ and } s$$

$$\text{R.H.S.} = \{\epsilon, r, rr, rs, sr, rsr, rrs, srr, \dots\}$$

$$= \epsilon + \text{any combination of } r \text{ and } s$$

As, L.H.S. = R.H.S.

$$(r^* s^*)^* = (r + s)^* \text{ is true.}$$

Example 2.5.11 Explain your answer in each of the following :

1) Every subset of a regular language is regular

2) Every regular language has a regular proper subset. **SPPU : May-06, Dec.-08, Marks 6**

Solution :

- 1) Every subset of regular language need not be regular. To support if consider a language.

$$L_1 = 0^* \text{ is regular but}$$

$$L_2 = \{0^{2n} \mid n \geq 1\} \text{ is not a regular language.}$$

where $L_2 \subseteq L_1$ Hence given statement is **false**.

- 2) Every regular language has a regular proper subset. To support this statement, consider a language.

$$L_1 = \phi \text{ it has no proper subset.}$$

$L_1 = \{\epsilon\}$ then $L_2 = \phi$ is a proper subset of L_1

L_1 = Language of any substring, has any substring which forms its proper subset.

Hence given statement is **true**.

Example 2.5.12 If $S = \{aa, b\}$ write all the string in S^* which are having length 4 or less, also say the following is True or False.

i) $(S^+)^* = (S^*)^*$ ii) $(S^+)^+ = S^+$ iii) $(S^*)^+ = (S^+)^*$

SPPU : May-09, Marks 4

Solution : $S = \{aa, b\}$

- 1) String having length 0 = $\{\epsilon\}$
- 2) Strings having length 1 = $\{b\}$
- 3) Strings having length 2 = $\{aa, bb\}$
- 4) Strings having length 3 = $\{aab, bbb, baa\}$
- 5) Strings having length 4 = $\{aaaa, aabb, bbbb, bbba, baab\}$

Thus we obtain the string with length 4 or less by combining all above obtained sets, $\{\epsilon, b, aa, bb, aab, bbb, baa, aaaa, aabb, bbbb, bbba, baab\}$

i) $(S^+)^* = (S^*)^*$

L.H.S. = $\{aa, b, aab, bb, bbb,\}^*$
 = $\{\epsilon + \text{any combination of } aa \text{ and } b\}$

R.H.S. = $\{\epsilon, aa, b, aab, bb, bbb,\}^*$
 = $\{\epsilon + \text{any combination of } aa \text{ and } b\}$

As L.H.S. = R.H.S. given statement is **true**.

ii) $(S^+)^+ = S^+$

L.H.S. = $\{aa, b, aab, bb, bbb,\}^+$
 = any combination of aa and b without ϵ .

R.H.S. = $\{aa, b, aab, bb, bbb,\}$
 = any combination of aa and b without ϵ .

As L.H.S. = R.H.S. given statement is **true**.

iii) $(S^*)^+ = (S^+)^*$

L.H.S. = $\{\epsilon + \text{any combination of } aa \text{ and } b\}^+$

R.H.S. = $\{\text{any combination of } aa \text{ and } b \text{ without } \epsilon\} + \epsilon$

$$\text{L.H.S.} = \text{R.H.S.}$$

Hence given statement is **true**.

Example 2.5.13 Give the examples of sets that demonstrate the following inequality. Here r_1, r_2, r_3 are regular expressions :

- 1) $r_1 + \epsilon \neq r_1$ 2) $r_1 \cdot r_2 \neq r_2 \cdot r_1$
 3) $r_1 \cdot r_1 \neq r_1$ 4) $r_1 + (r_2 \cdot r_3) \neq (r_1 + r_2) \cdot (r_1 r_3)$.

SPPU : Dec.-09, Marks 10

Solution : Assume $r_1 = a, r_2 = b, r_3 = ab$

- 1) L.H.S. = $r_1 + \epsilon = \{ a, \epsilon \}$
 R.H.S. = $r_1 = \{a\}$
 $\therefore$ L.H.S. $\neq$ R.H.S.
- 2) L.H.S. = $r_1 r_2 = ab$
 R.H.S. = $r_2 r_1 = ba$
 $\therefore$ L.H.S. $\neq$ R.H.S.
- 3) L.H.S. = $r_1 r_1 = aa$
 R.H.S. = $r_1 = a$
 $\therefore$ L.H.S. $\neq$ R.H.S.
- 4) L.H.S. = $r_1 + (r_2 r_3)$
 = $a + (b \cdot ab)$
 = $a + bab = \{a, bab\}$
 R.H.S. = $(r_1 + r_2) \cdot (r_1 r_3)$
 = $(a + b) (a \cdot ab)$
 = $\{aaab, baab\}$
 $\therefore$ L.H.S. $\neq$ R.H.S.

Example 2.5.14 Check the equivalence of the regular expressions.

- 1) $(a^* bbb)^* a^*$ AND $a^* (bbb a^*)^*$
 2) $((a + bb)^* aa)^*$ AND $\epsilon + (a + bb)^*.aa$

SPPU : May-11, Marks 4

Solution :

- 1) $(a^* bbb)^* a^*$ AND $a^* (bbb a^*)^*$
 By identity rule $(PQ)^* P = P (QP)^*$

Let us map this rule with given expression

$$(a^* b b b)^* a^* = a^* (b b b a^*)^*$$

$\begin{array}{ccccccc} \downarrow & & \downarrow & & \downarrow & & \downarrow \\ P & & Q & & P & & P \\ & & & & & & \downarrow \\ & & & & & & Q \end{array}$

Hence given expressions are equivalent

2) $((a+bb)^* aa)^*$ AND $\epsilon + (a+bb)^*.aa$

$$\begin{aligned} \text{L.H.S.} &= ((a+bb)^* aa)^* \\ &= \{\epsilon, aaa, bbaa, aaaa, bbaaaa, abbaa, \dots\} \end{aligned}$$

$$\begin{aligned} \text{R.H.S.} &= \epsilon + (a+bb)^* aa \\ &= \{\epsilon, aaa, bbaa, aaaa, bbaaaa, abbaa, \dots\} \end{aligned}$$

$$\text{L.H.S.} = \text{R.H.S.}$$

Hence given expressions are equivalent.

2.6 RE to DFA

SPPU : May-10, 12, 14, 15, 19, Dec.-11, 13, 14, 16, 18, 19, Aug.-15, 17, Oct.-18, 19, Marks 10

Regular expression can be converted to FA using two methods -

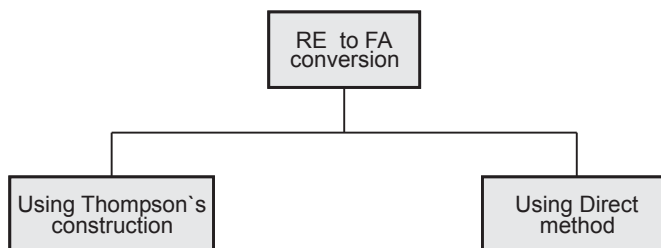


Fig. 2.6.1 Methods of conversion

1. Thomson's Construction

- There are some patterns of regular expressions for which the finite automata is constructed.
- These patterns are -

Case 1 : Zero operators

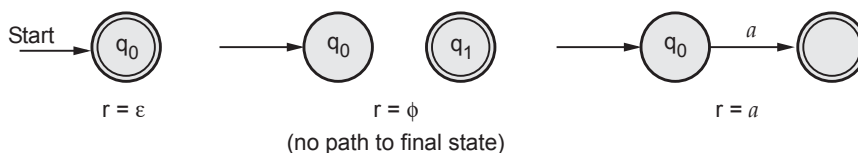
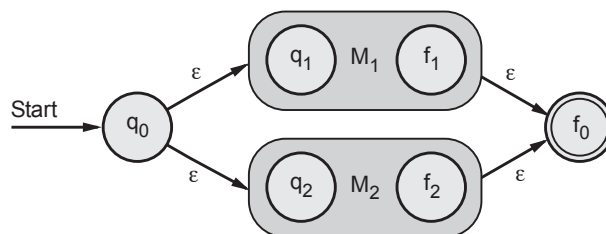
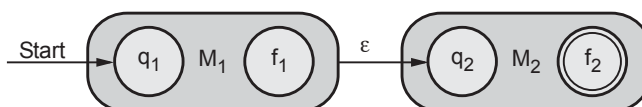


Fig. 2.6.2 Finite automata for given regular expression

Case 2 : Union case**Fig. 2.6.3 The machine M for union**

Let $r = r_1 + r_2$ where r_1 and r_2 be the regular expressions.

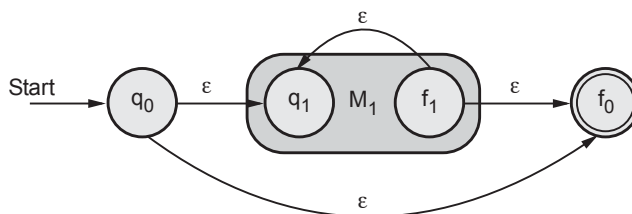
Case 3 : Concatenation case**Fig. 2.6.4 Machine M for concatenation**

$r = r_1 r_2$ be regular expression

Case 4 : Closure case

$r = r_1^*$ be regular expression

The machine M will be

**Fig. 2.6.5****• Direct Method**

- This method is a direct method for obtaining FA from given regular expression. This is called a **subset** method. The method is given as below -

Step 1 : Design a transition diagram for given regular expression, using NFA with ϵ moves.

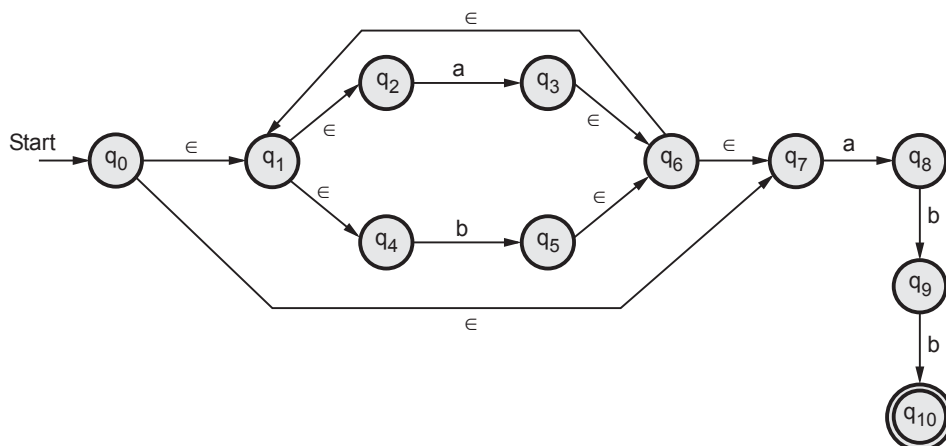
Step 2 : Convert this NFA with ϵ to NFA without ϵ .

Step 3 : Convert the obtained NFA to equivalent DFA.

Example 2.6.1 Construct DFA for the Regular Expression $(a + b)^* abb$.

SPPU : May-12, Marks 6, Aug.-15, In Sem, Marks 8

Solution : We will first construct NFA - ϵ for given r.e.



ϵ - closure (q_0) = $\{q_1, q_2, q_4, q_7\}$ = Call it as state A.

$$\begin{aligned}\delta'(A, a) &= \epsilon\text{-closure}(\delta(q_1, q_2, q_4, q_7), a) \\ &= \epsilon\text{-closure}(q_3, q_8) \\ &= \{q_1, q_2, q_3, q_4, q_6, q_7, q_8\} = \text{Call it as state B.}\end{aligned}$$

$\therefore \delta'(A, a) = \mathbf{B}$

$$\begin{aligned}\delta'(A, b) &= \epsilon\text{-closure}(\delta(q_1, q_2, q_4, q_7), b) \\ &= \epsilon\text{-closure}(q_5) = \{q_1, q_2, q_4, q_5, q_6, q_7\}\end{aligned}$$

$\delta'(A, a) = \mathbf{C}$

$$\begin{aligned}\delta'(B, a) &= \epsilon\text{-closure}(\delta(q_1, q_2, q_3, q_4, q_6, q_7, q_8), a) \\ &= \epsilon\text{-closure}(q_3, q_8)\end{aligned}$$

$\delta'(B, a) = \mathbf{B}$

$$\begin{aligned}(B, b) &= \epsilon\text{-closure}(\delta(q_1, q_2, q_3, q_4, q_6, q_7, q_8), b) \\ &= \epsilon\text{-closure}(q_5, q_9) \\ &= \{q_1, q_2, q_4, q_5, q_6, q_7, q_9\}\end{aligned}$$

$\delta'(B, b) = \mathbf{D}$

$$\begin{aligned}\delta'(C, a) &= \epsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7), a) \\ &= \epsilon\text{-closure}(q_3, q_8)\end{aligned}$$

$$\delta'(C, a) = B$$

$$\begin{aligned}\delta'(C, b) &= \varepsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7), b) \\ &= \varepsilon\text{-closure}(q_5)\end{aligned}$$

$$\delta'(C, b) = C$$

$$\begin{aligned}\delta'(D, a) &= \varepsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7, q_9), a) \\ &= \varepsilon\text{-closure}(q_3, q_8)\end{aligned}$$

$$\delta'(D, a) = B$$

$$\begin{aligned}\delta'(D, b) &= \varepsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7, q_9), b) \\ &= \varepsilon\text{-closure}(q_5, q_{10}) \\ &= \{q_1, q_2, q_4, q_5, q_6, q_7, q_{10}\}\end{aligned}$$

$$\delta'(D, b) = E$$

$$\begin{aligned}\delta'(E, a) &= \varepsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7, q_{10}), a) \\ &= \varepsilon\text{-closure}(\{q_3, q_8\})\end{aligned}$$

$$\delta'(E, a) = B$$

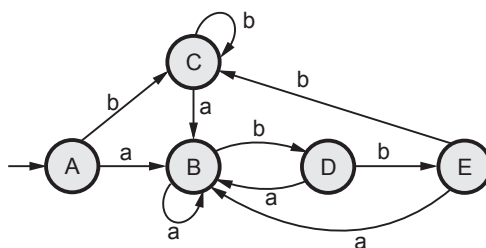
$$\begin{aligned}\delta'(E, b) &= \varepsilon\text{-closure}(\delta(q_1, q_2, q_4, q_5, q_6, q_7, q_{10}), b) \\ &= \varepsilon\text{-closure}(q_5)\end{aligned}$$

$$\delta'(E, b) = C$$

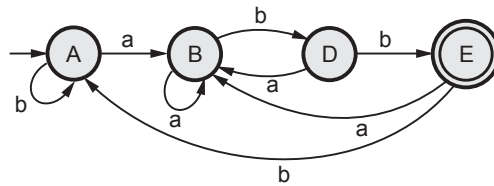
The transition table will be

State \ I/P		
	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

The transition diagram will be



The states A and C are equivalent. Hence we can have minimized DFA as

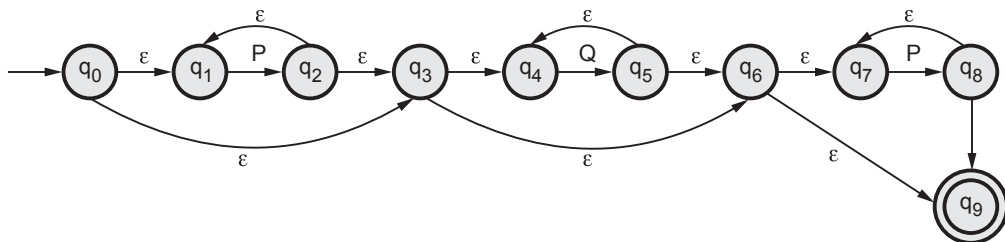


Example 2.6.2 Show that $P^*(QP^*)^* = (P + Q)^*$

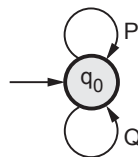
SPPU : May-14, Marks 4

Solution : We will show this equality using FA from R.E.

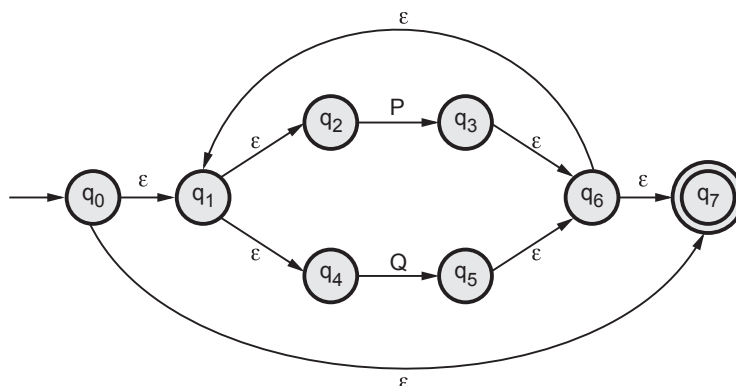
Step 1 : L.H.S. = $P^*(QP^*)^*$. The NFA with ϵ for this regular expression will be



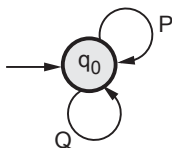
On eliminating ϵ moves we get -



Step 2 : R.H.S. = $(P + Q)^*$



On eliminating ϵ moves we get -



From Step 1 and Step 2 $P^*(QP^*)^* = (P + Q)^*$ true.

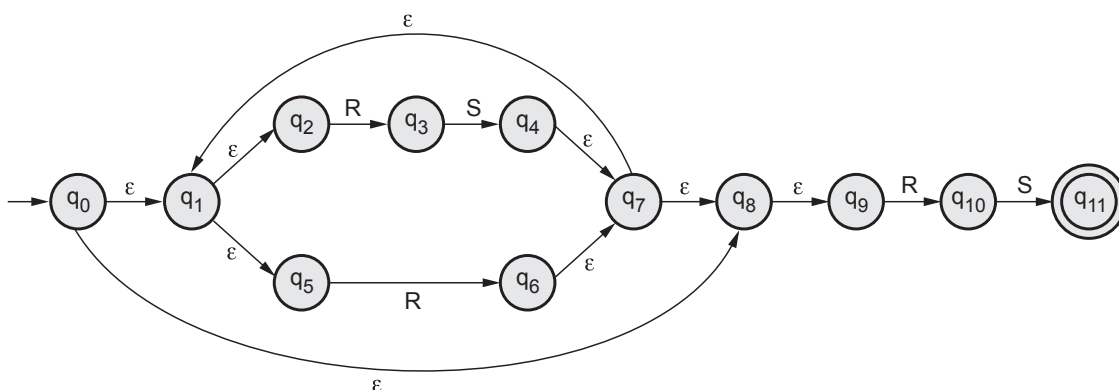
Example 2.6.3 Prove or disprove each of the following about regular expression :

- 1) $(RS + R)^*RS = (RR^*S)^*$ 2) $(R + S)^* = R^* + S^*$

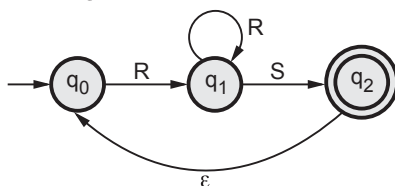
SPPU : Dec.-13, Marks 4

Solution :

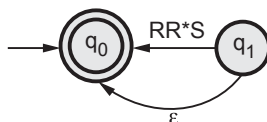
1) Step 1 : L.H.S. = $(RS + R)^* RS$



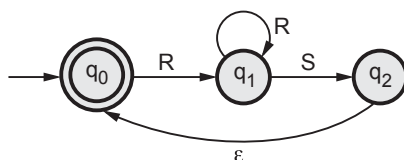
On eliminating ϵ transitions we get -



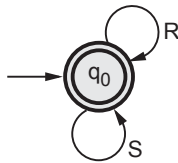
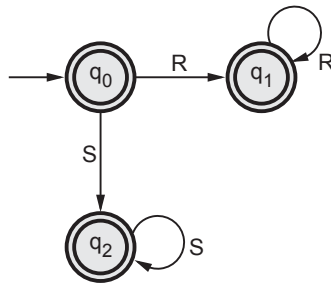
Step 2 : R.H.S. = $(RR^*S)^*$



As L.H.S. and R.H.S. both does not yield same states (change in final state)



$(RS + R)^* RS \neq (RR^*S)^*$

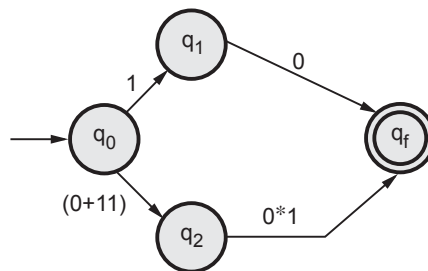
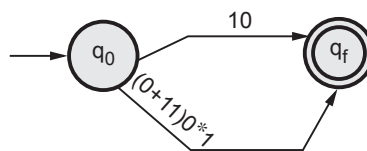
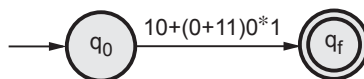
2) Step 1 : $(R + S)^*$ **Step 2 :** $R^* + S^*$ 

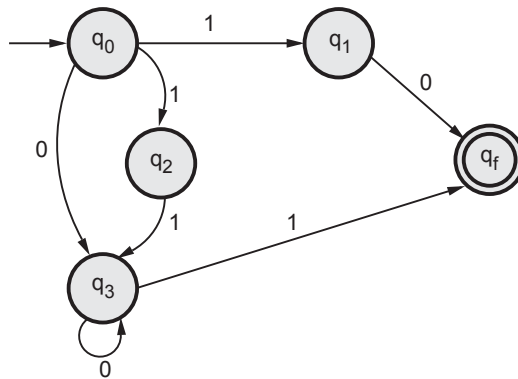
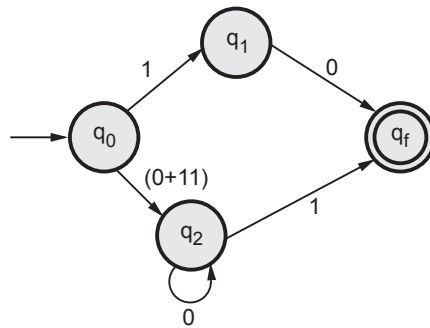
This proves that $(R + S)^* \neq (R^*S^*)^*$

Example 2.6.4 Design a FA from given regular expression $10 + (0 + 11)0^*1$.

SPPU : Dec.-11, Marks 10

Solution : First we will construct the transition diagram for given regular expression.





Now we have got NFA without ϵ . Now we will convert it to required DFA for that, we will first write a transition table for this NFA.

State \ Input	Input	
	0	1
q ₀	q ₃	{q ₁ , q ₂ }
q ₁	q _f	ϕ
q ₂	ϕ	q ₃
q ₃	q ₃	q _f
q _f	ϕ	ϕ

The equivalent DFA will be

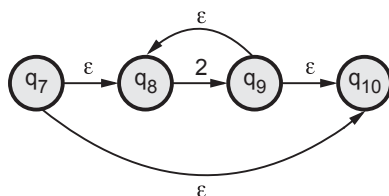
State \ Input	0	1
$[q_0]$	$[q_3]$	$[q_1, q_2]$
$[q_1]$	$[q_f]$	ϕ
$[q_2]$	ϕ	$[q_3]$
$[q_3]$	$[q_3]$	$[q_f]$
$[q_1, q_2]$	$[q_f]$	$[q_3]$
$[q_f]$	ϕ	ϕ

Example 2.6.5 Construct minimized DFA accepting language represented by regular expression $0^*1^*2^*$. Convert given regular expression to NFA with ϵ moves and then convert NFA with ϵ moves to DFA using direct method.

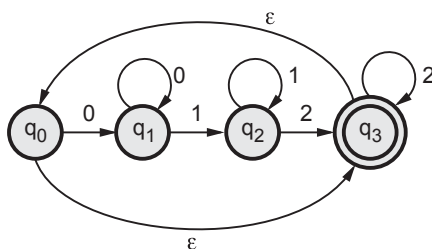
SPPU : Dec.-13, Marks 8, Oct.-19, In Sem, Marks 4

Solution :

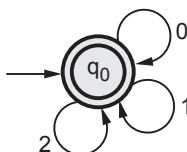
Step 1 :



Step 2 : Byeliminating ϵ moves



Step 3 : The DFA will be



Example 2.6.6 Find FA for a given RE : $(0 + 1)^* \cdot (010 + 101) \cdot (0 + 1)^*$.

SPPU : Dec.-14, End Sem, Marks 4

Solution : The language L contains substring 010 or 101. Hence FA will be,

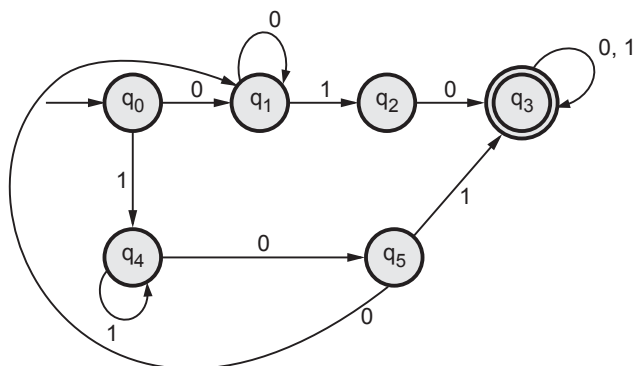
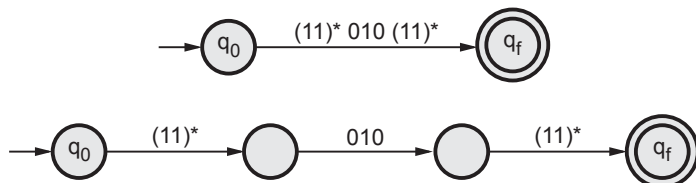


Fig. 2.6.6

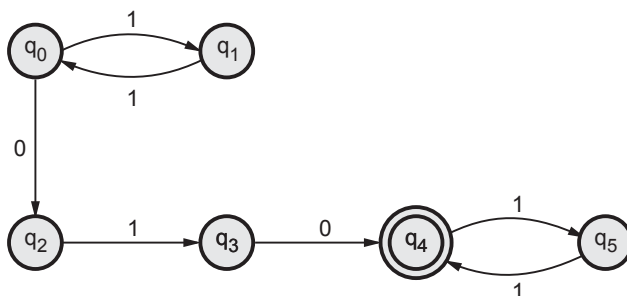
Example 2.6.7 Construct FA for the regular expression : $(11)^* \cdot 010 \cdot (11)^*$.

SPPU : May-15, End Sem, Marks 4

Solution :



The FA will be :



Example 2.6.8 Design a Moore machine and Mealy to generate 1's complement of given binary number.

SPPU : May-10,15, End Sem, Marks 6

Solution : To generate 1's complement of given binary number the simple logic which we will apply is that if input is 0 then output will be 1 and if input is 1 then output will be 0. That means there are three state one-start state, second state is for taking 0's as input and produces output as 1. Then third state is for taking 1's as input and producing output as 0.

Hence the Moore machine will be as shown in Fig. 2.6.7.

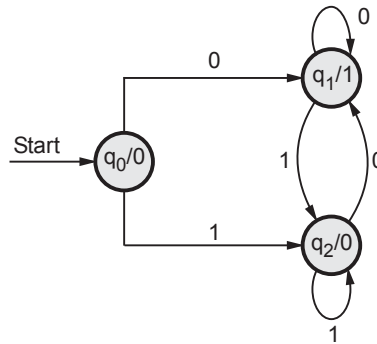


Fig. 2.6.7

For instance, take one binary number 1011 then

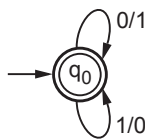
Input		1	0	1	1
State	q ₀	q ₂	q ₁	q ₂	q ₂
Output	0	0	1	0	0

Thus we get 00100 as 1's complement of 1011, we can neglect the initial 0 and the output which we get is 0100 which is 1's complement of 1011. The transition table can be drawn as below -

Current state	Next state		Output
	0	1	
q ₀	q ₁	q ₂	0
q ₁	q ₁	q ₂	1
q ₂	q ₁	q ₂	0

Thus Moore machine $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0)$;
 where $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $\Delta = \{0, 1\}$.

The transition table shows the δ and λ functions.

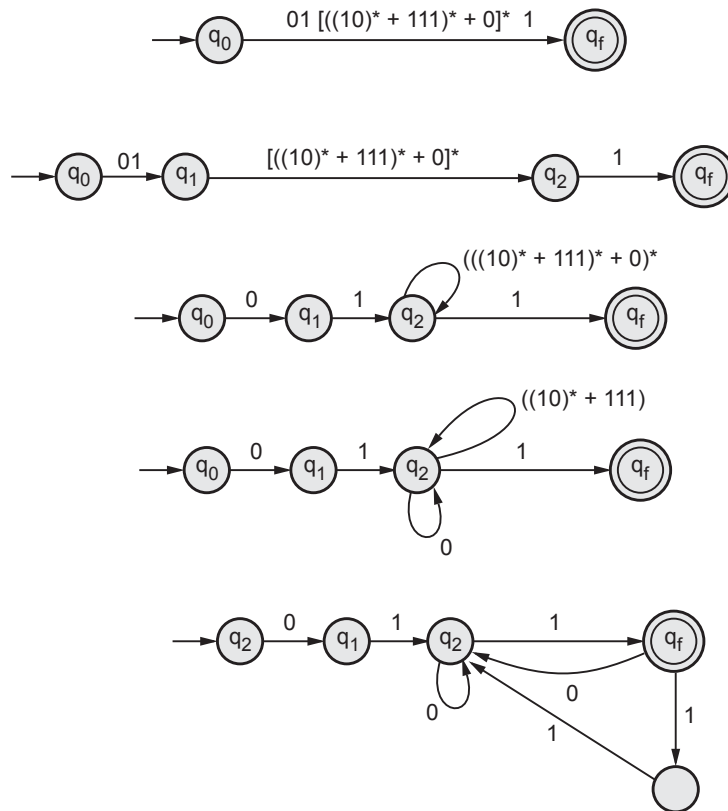
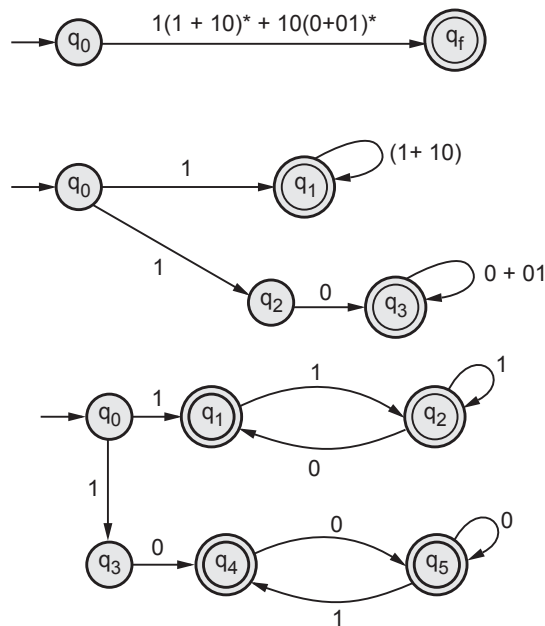


Mealy machine

Example 2.6.9 Construct finite automata for the following regular expressions :

- $01[(10)^* + 111]^* + 0]^* 1$
- $1(1+10)^* + 10(0+01)^*$

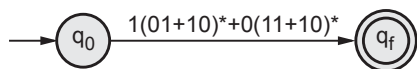
SPPU : Dec.-16, End Sem, Marks 6

Solution :**i)****ii)**

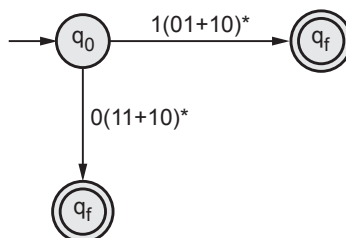
Example 2.6.10 Construct the finite Automata defined over $\Sigma = \{0, 1\}$ for the following regular expression $1(01+10)^* + 0(11+10)^*$

SPPU : Aug.-17, In Sem, Marks 3

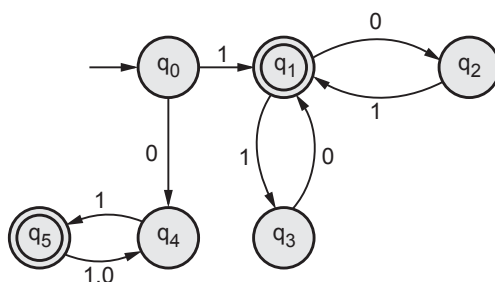
Solution : Step 1 :



Step 2 :



Step 3 :



Example 2.6.11 Convert the following regular expression to ϵ -NFA and find the ϵ -closure of all the states. $(0+1)^* \cdot 1 \cdot (0+1)$

SPPU : Aug.-17, In Sem, Marks 3

Solution :

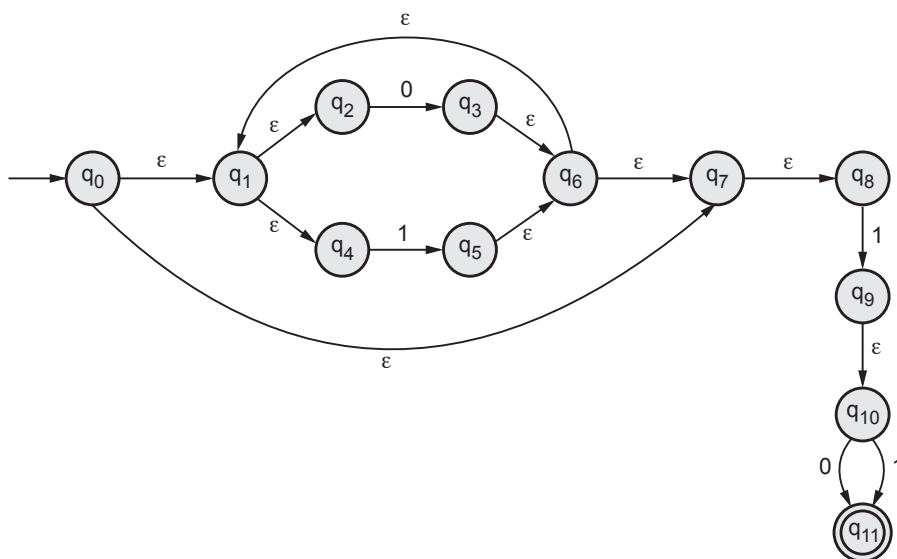


Fig. 2.6.8

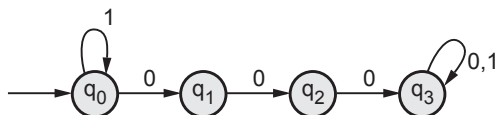
ϵ - closure (q_0) = $\{q_0\}$
 ϵ - closure (q_1) = $\{q_1, q_2, q_4\}$
 ϵ - closure (q_2) = $\{q_2\}$
 ϵ - closure (q_3) = $\{q_3, q_6, q_1, q_2, q_4, q_7, q_8\}$
 ϵ - closure (q_4) = $\{q_4\}$
 ϵ - closure (q_5) = $\{q_5, q_6, q_1, q_2, q_4, q_7, q_8\}$
 ϵ - closure (q_6) = $\{q_6, q_7, q_8, q_1, q_2, q_4\}$
 ϵ - closure (q_7) = $\{q_7, q_8\}$
 ϵ - closure (q_8) = $\{q_8\}$
 ϵ - closure (q_9) = $\{q_9\}$
 ϵ - closure (q_{10}) = $\{q_{10}\}$
 ϵ - closure (q_{11}) = $\{q_{11}\}$

Example 2.6.12 Design FA accepting the following language over the alphabet $\{0, 1\}$

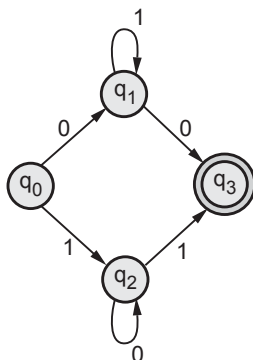
- Set of all strings having at least three consecutive zeros.
- Set of all strings that begin and end with same symbol.

SPPU : Dec.-18, In Sem, Marks 8

Solution : i)



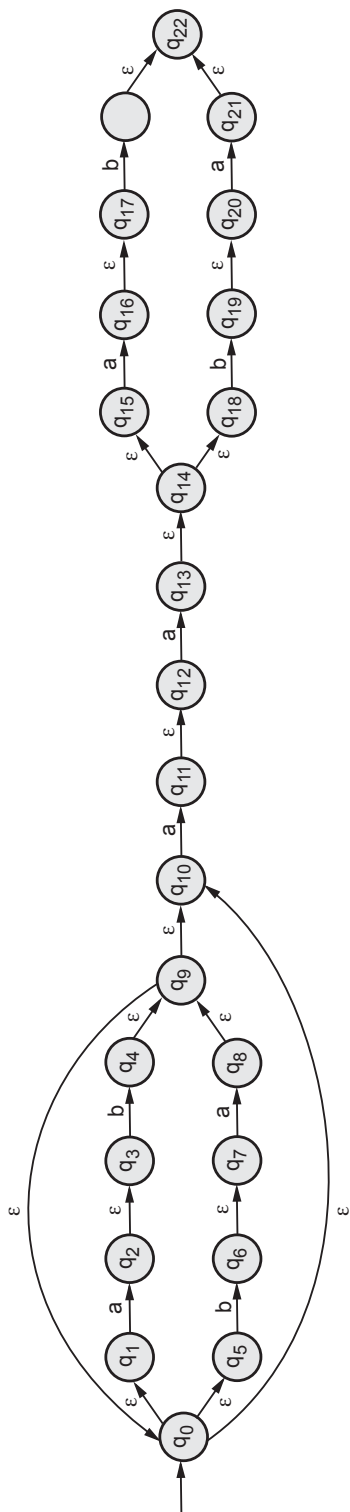
ii)



Example 2.6.13 For the following regular expression defined over $\Sigma = \{a, b\}$, draw ϵ - NFA recognizing the corresponding language $(ab + ba)^* aa (ab + ba)$.

SPPU : Oct.-18, In Sem, Marks 4

Solution :



Example 2.6.14 Show that : i) $(a.b)^* \neq a^* \cdot b^*$ ii) $(a+b)^* = (a+b)^* \cdot (a+b)^*$

SPPU : May-19, End Sem, Marks 6

Solution : i) $(a.b)^* \neq a^* \cdot b^*$

1) Let us represent $(ab)^*$ by FA

L.H.S.

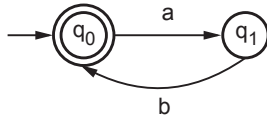


Fig. 2.6.9

$L = \{^{\wedge}, ab, abab, ababab \dots\}$

2) Let us represent $a^* b^*$

R.H.S.

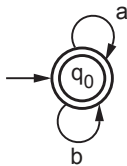


Fig. 2.6.10

$L = \{^{\wedge}, a, aa, aaa, \dots, b, bb, \dots, ab, abab, ba, baba, \dots\}$

From (1) and (2) L.H.S. $\neq$ R.H.S.

Hence $(ab)^* \neq a^* b^*$

ii) $(a+b)^* = (a+b)^* \cdot (a+b)^*$

Let, R.H.S. be $(a+b)^* + (a+b)^* = (a+b)^* \because R + R = R$

L.H.S.

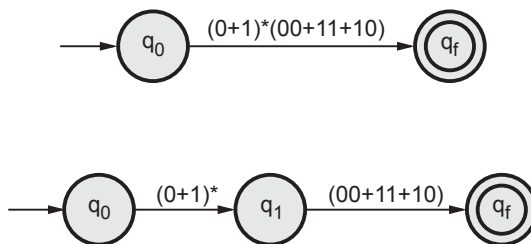
Hence $(a+b)^* = (a+b)^* + (a+b)^*$

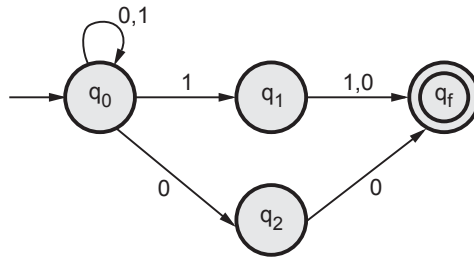
Example 2.6.15 Construct a finite automata for the following regular expression.

i) $(1+0)^* (00+11+10)$ ii) $1(01+10)^* 0(11+10)^*$ **SPPU : Dec.-19, End Sem, Marks 8**

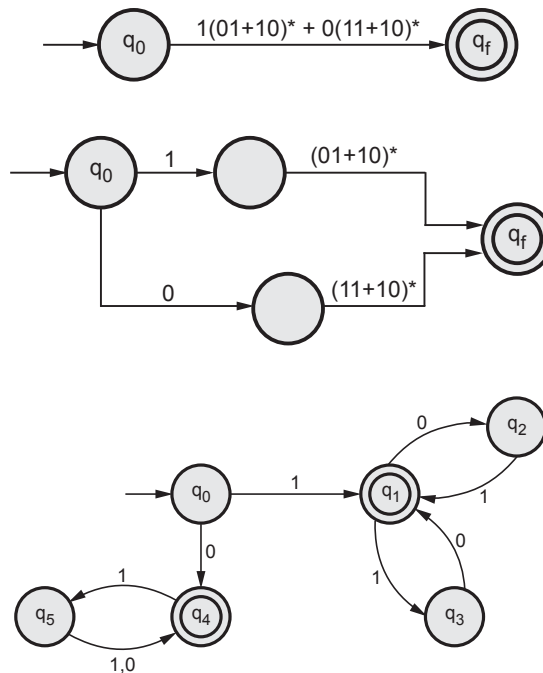
Solution :

i) r.e. = $(1+0)^* (00+11+10)$





ii) $\text{r.e.} = 1(01+10)^* + 0(11+10)^*$



2.7 DFA to RE using Arden's Theorem

SPPU : May-09, 10, 14, 15, 16, Dec.-08, 09, 10, 13, 14, 15, Aug.-15, Marks 10

There are two methods for conversion of DFA to Regular Expression.

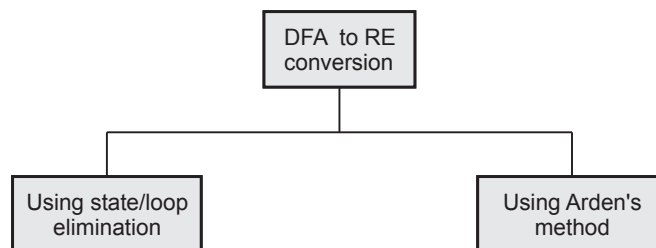


Fig. 2.7.1 DFA to RE conversion methods

2.7.1 State / Loop Elimination

- 1) The state elimination is the simplest method in which the states of DFA are removed one by one. Until we left with one start and one final state, for each removed state regular expression is generated.
- 2) In this method, each newly generated regular expression acts as input for a state which is next to removed state.
- 3) The advantage of this method is it becomes easier to visualize regular expression from the states of FA.

Example 2.7.1 Convert the following FA to RE using state elimination method.

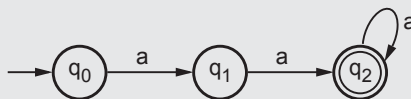


Fig. 2.7.2

Solution :

Step 1 :

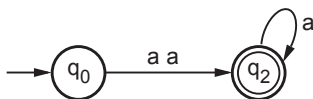


Fig. 2.7.3

Step 2 :



Fig. 2.7.4

Step 3 : R.E. = aaa^*

Example 2.7.2 Obtain RE from given FA using loop /state elimination method.

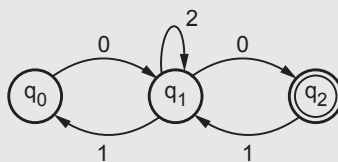


Fig. 2.7.5

Solution :

Step 1 :

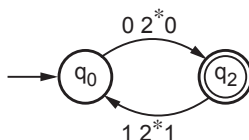
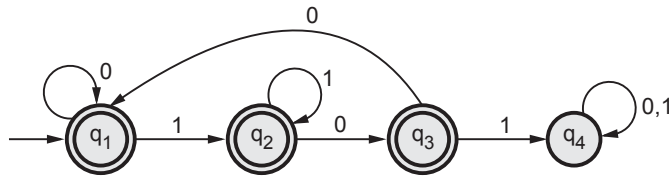


Fig. 2.7.6

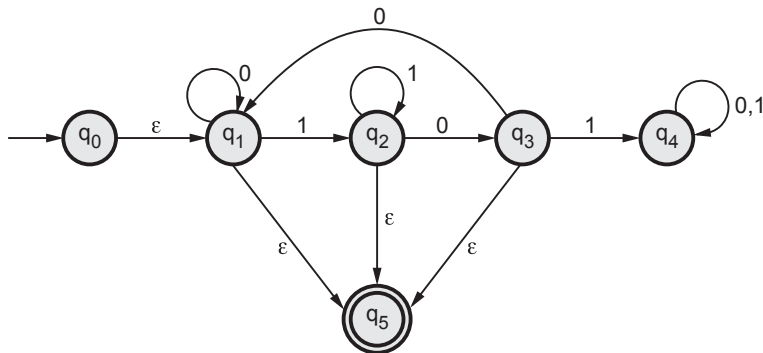
Step 2 : R.E. = $[(02^*0)(12^*1)]^*$

Example 2.7.3 Construct NFA and DFA for accepting all possible string of 0's and 1's not containing 101 as substring. Find regular expression for it. **SPPU : May-14, Marks 8**

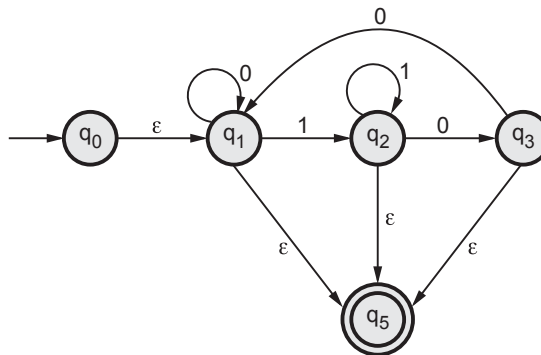
Solution : The DFA for the given language is



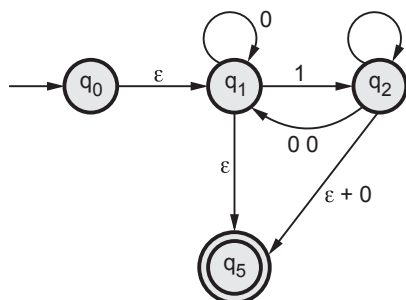
Now we will convert this DFA to NFA with ϵ . We will add one start state q_0 and new final state i.e. q_5 .



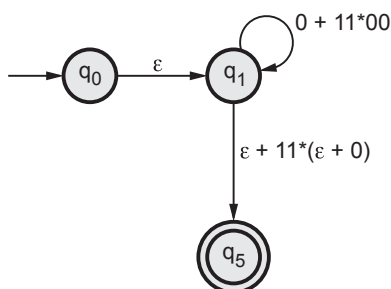
Now we will eliminate state q_4 . The NFA will be -



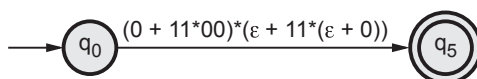
We will eliminate state q_3 , without changing meaning of it.



We will eliminate state q_2 .



Now we will eliminate state q_1 . The NFA will be -



The regular expression will be $(0 + 11^*00)^* (\epsilon + 11^* (\epsilon + 0))$

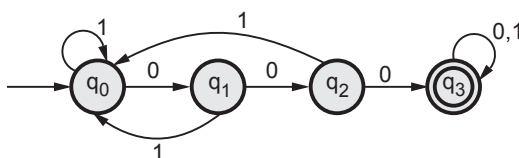
Example 2.7.4 Find the regular expression corresponding to each of the following subset of $\{0,1\}^*$

- The language of all strings not containing the substring 000
- The language of all strings that do not contain the substring 110
- The language of all strings containing both 101 and 010 as substring.

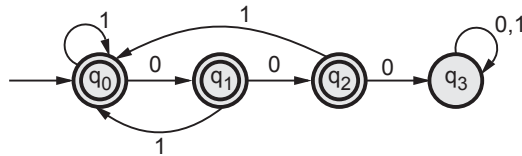
SPPU : May-10, Marks 10

Solution :

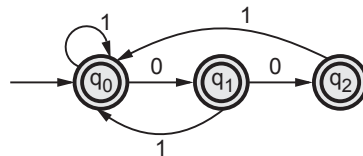
- To obtain r.e. for this L we will first design FA for strings containing 000 as substring.



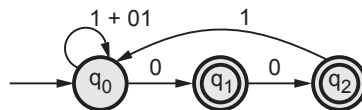
Now to obtain FA for strings not containing 000 as substring just make final state of above FA as non-final and non-final states as final state. The FA will then be -



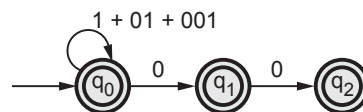
Clearly in above FA q_3 is a dead state. So eliminate it.



Now we will eliminate the edge between q_0 and q_1 which is giving $(01)^*$

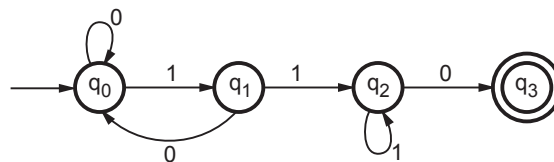


Similarly, we will eliminate edge from q_2 to q_0 giving $(001)^*$

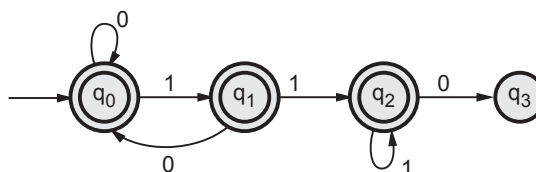


This generates **r.e. = $(1+01+001)^* [\epsilon + 0 + 00]$**

- b)** Let us draw the transition graph for the language that contains 110.



Now for the language that do not contain 110, we will change final states to non-final states and non-final states to final state.



From this diagram, the regular expression will be.

$$\text{r.e.} = [0^* (10)^* + 1^*]$$

c) Regular expression = $[(0+1)^* 101 (0+1)^* 110 (0+1)^*]$
 $+ (0+1)^* 110(0+1)^* 101(0+1)^*]$

2.7.2 Arden's Theorem

Conversion Method

Following algorithm is used to build the r.e. from given DFA.

1. Let q_1 be the initial state.
2. There are $q_2, q_3, q_4, \dots, q_n$ number of states. The final state may be some q_j where $j \leq n$.
3. Let α_{ji} represents the transition from q_j to q_i .
4. Calculate q_i such that

$$q_i = \alpha_{ji} \cdot q_j$$

If q_i is a start state

$$q_i = \alpha_{ji} q_j + \varepsilon$$

5. Similarly compute the final state which ultimately gives the regular expression r .

Example 2.7.5 Construct r.e. from given DFA.

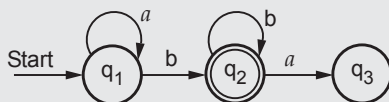


Fig. 2.7.7

Solution : Let us write down the equations

$$q_1 = q_1 a + \varepsilon$$

Since q_1 is a start state ε will be added and the input a is coming to q_1 from q_1 hence we write

$$\text{State} = \text{Source state of input} \times \text{Input coming to it}$$

Similarly $q_2 = q_1 b + q_2 b$

Let us simplify q_1 first

$$q_1 = q_1 a + \varepsilon$$

We can re-write it as

$$q_1 = \varepsilon + q_1 a$$

which is similar to $R = Q + RP$ which further gets reduced to $R = QP^*$.

Assuming $R = q_1$, $Q = \epsilon$, $P = a$

We get $q_1 = \epsilon \cdot a^*$

$$q_1 = a^* \quad \because \quad \epsilon \cdot R^* = R^*$$

Substituting value of q_1 in q_2 we get

$$q_2 = q_1 b + q_2 b$$

$$q_2 = a^* b + q_2 b$$

We can compare this equation with $R = Q + RP$ assuming $R = q_2$, $Q = a^*b$, $P = b$ which gets reduced to $R = QP^*$.

$$\therefore \quad q_2 = a^*b \cdot b^*$$

$$\text{As } R R^* = R^+$$

$$q_2 = a^* \cdot b^+$$

From the given DFA, if we want to find out the regular expression, we normally calculate the equation for final state. Since in the given DFA q_2 is a final state and $q_2 = a^* b^+$. We can conclude as the DFA represents $a^* b + b^+$ as regular expression.

Example 2.7.6 Find out the regular expression from given DFA.

SPPU : Dec.-15, End Sem, Marks 6

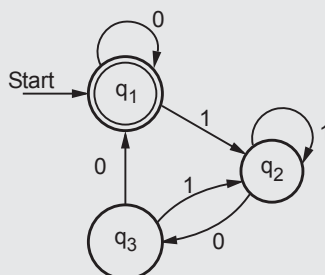


Fig. 2.7.8

Solution : Let us solve the DFA by writing the regular expression, for each state.

$$q_1 = q_1 0 + q_3 0 + \epsilon \quad \because \text{Initial state}$$

$$q_2 = q_2 1 + q_3 1 + q_1 1$$

$$q_3 = q_2 0$$

For getting the r.e. we have to solve q_1 the final state.

$$q_2 = q_2 1 + q_2 0 1 + q_1 1$$

$$q_2 = q_2 (1 + 01) + q_1 1$$

We will compare $R = Q + R P$ with above equation, so $R = q_2$, $Q = q_1 1$, $P = (1 + 01)$ which ultimately gets reduced to $Q P^*$.

$$q_2 = q_1 1 (1 + 01)^*$$

Substituting this value to q_1

$$\begin{aligned} q_1 &= q_1 0 + q_2 1 + \varepsilon \\ &= q_1 0 + q_2 1 + \varepsilon \\ &= q_1 0 + q_1 (1 (1 + 01)^*) 1 + \varepsilon \\ q_1 &= q_1 (0 + 1 (1 + 01)^*) + \varepsilon \end{aligned}$$

Again $R = Q + R P$

where $R = q_1$

$$Q = \varepsilon$$

$$P = 0 + 1 (1 + 01)^* 00$$

Hence $q_1 = \varepsilon \cdot [0 + 1 (1 + 01)^* \cdot 00]^*$

$$q_1 = [0 + 1 (1 + 01)^* \cdot 00]^* \quad \because \varepsilon \cdot R = R \text{ is required r.e.}$$

Example 2.7.7 Represent the language given by following DFA.

SPPU : Dec.-14, End Sem, Marks 4

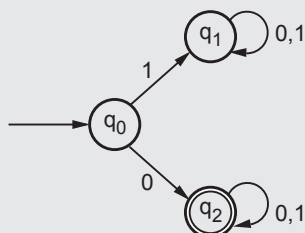


Fig. 2.7.9

Solution : To find the language, we will solve for regular expression,

$$q_0 = \varepsilon$$

$$q_2 = q_2 0 + q_2 1 + q_0 0$$

Substituting the value of q_0 in q_2

$$q_2 = q_2 (0 + 1) \varepsilon \cdot 0$$

$$q_2 = q_2 (0 + 1) + 0 \quad \because R = Q + R P \Rightarrow Q P^*$$

$$q_2 = 0 (0 + 1)^* \quad \text{where } Q = 0, P = (0 + 1), R = q_2$$

This is a final regular expression, which represents the language containing all the strings which start with letter 0.

Example 2.7.8 Describe in english the language indicated by following regular expression.

SPPU : Dec.-10, Marks 6

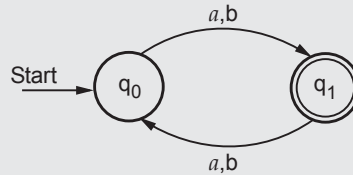


Fig. 2.7.10

Solution : The equations for the states are

$$q_0 = q_1 a + q_1 b + \epsilon \quad \dots(1)$$

$$q_1 = q_0 (a + b) \quad \dots(2)$$

We will simplify q_0 first

$$q_0 = (a + b) q_1 + \epsilon$$

Let us put this value of q_0 into equation (2).

$$q_1 = ((a+b)q_1 + \epsilon)(a+b)$$

$$q_1 = (a+b)(a+b)q_1 + (a+b)$$

This is equivalent to $R = Q + R P$ where $R = q_1$, $Q = (a+b)$, $P = (a+b)(a+b)$ which gets reduced to $Q P^*$.

$$\therefore q_1 = (a+b) \cdot [(a+b)(a+b)]^*$$

This is a regular expression indicated by DFA.

The language given by this DFA is the language in which it accepts all the strings of odd length.

Example 2.7.9 Consider the FA and construct regular expressions that is accepted by it. Fig. 2.7.11.

SPPU : Dec.-13, Marks 6

Solution : We will first write the equations for each state, assuming q_0 as initial state

$$q_0 = q_0 0 + q_2 0 + q_3 0 + \epsilon \quad \dots (1)$$

$$q_1 = q_0 1 + q_1 1 + q_3 1 \quad \dots (2)$$

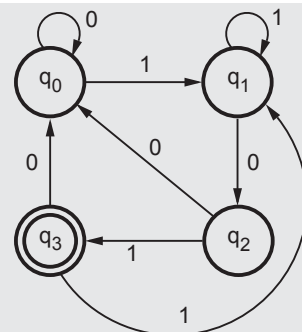


Fig. 2.7.11

$$q_2 = q_1 0 \quad \dots (3)$$

$$q_3 = q_2 1 \quad \dots (4)$$

Now we will place value of q_2 in equation (4)

$$q_3 = q_1 01 \quad \dots (5)$$

Put value of equation (5) in equation (2)

$$q_1 = q_0 1 + q_1 1 + (q_1 01) 1$$

$$q_1 = q_0 1 + q_1 1 + q_1 011$$

$$\text{gives us } \underbrace{q_1}_R = \underbrace{q_1}_R \underbrace{(1 + 011)}_P + \underbrace{q_0 1}_Q$$

$$q_1 = q_0 1 (1 + 011)^* \quad \because R = Q + RP \text{ gives } R = QP^*$$

Now solving q_0

$$q_0 = q_0 0 + q_2 0 + q_3 0 + \varepsilon$$

$$= q_0 0 + (q_1 0) 0 + (q_1 01) 0 + \varepsilon$$

$$q_0 = q_0 0 + q_1 (00 + 010) + \varepsilon$$

$$= q_0 0 + q_0 1 (1 + 011)^* (00 + 010) + \varepsilon$$

$$q_0 = q_0 (0 + 1 (1 + 011)^* (00 + 010)) + \varepsilon$$

$$\therefore q_0 = \varepsilon (0 + 1 (1 + 011)^* (00 + 010))^*$$

Now we will solve

$$q_3 = q_1 01$$

$$= q_0 1 (1 + 011)^* 01$$

$$q_3 = (0 + 1 (1 + 011)^* (00 + 010))^* 1 (1 + 011)^* 01.$$

As q_3 is a final state, we get the r.e. as equation for q_3 .

$$\therefore \text{r.e.} = (0 + 1 (1 + 011)^* (00 + 010))^* 1 (1 + 011)^* 01)$$

Example 2.7.10 Design the finite automata and then equivalent regular expression using Arden's theorem that accepts the set of all strings over the alphabet $\{a, b\}$ with an equal number of a 's and b 's, such that each prefix has atmost one more a than b 's and atmost one more b than a 's.

SPPU : Dec.-08, 09; May-09, Marks 10

Solution : The FA will be as shown in Fig. 2.7.12.

The valid strings 0101 contains prefix of 1 as 010 in which number of 0's are more than 1. Similarly in a valid string 1010 the prefix of 0 contains more number of 1's than 0's .

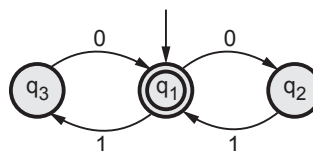


Fig. 2.7.12

Now in order to obtain the regular expression for this FA, we will write state equations as follows -

$$q_1 = q_2 1 + q_3 0 + \epsilon \quad \dots (1)$$

$$q_1 = q_1 0 \quad \dots (2)$$

$$q_3 = q_1 1 \quad \dots (3)$$

There is only one final state and that is q_1 . Hence equation of q_1 state is the regular expression.

$$\begin{array}{cccc}
 q_1 & = & q_1 & (01 + 10) + \epsilon \\
 \downarrow & & \downarrow & \quad \downarrow \quad \downarrow \\
 R & & R & \quad P \quad Q
 \end{array}$$

$\therefore$ Applying Arden's theorem.

Let,

$$q_1 = q_2 1 + q_3 0 + \epsilon$$

$$= q_1 01 + q_1 10 + \epsilon$$

$$q_1 = \epsilon (01 + 10)^*$$

$$q_1 = (01 + 10)^*$$

$\therefore$ Put equations 2 and 3 in equation 1

$$\therefore R = Q + RP \text{ is } R = QP^*$$

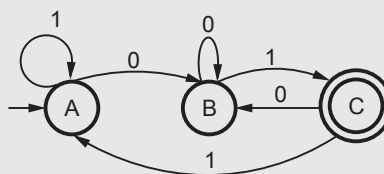
$$\therefore \epsilon R^* = R^*$$

Hence regular expression for given language is

$$\text{r.e.} = (01 + 10)^*$$

Example 2.7.11 Convert the following finite automation into its equivalent regular expression using Arden's Theorem.

SPPU : May-15, End Sem, Marks 6



$$\text{Solution : } A = A1 + C1 + \epsilon \quad \dots (1)$$

$$B = B0 + A0 + C0 \quad \dots (2)$$

$$C = B1 \quad \dots (3)$$

Putting equation (3) in equation (1), we get -

$$A = A1 + B11 + \epsilon$$

$$A = A1 + B11 + \varepsilon$$

$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ R & P & Q \end{array}$

$$\therefore A = (B11 + \varepsilon)1^* \quad \dots (4)$$

Now, put equation (3) in equation (2),

$$B = B0 + A0 + B10 \quad \dots (5)$$

put equation (4) in equation (5),

$$\begin{aligned} B &= B0 + (B11 + \varepsilon)1^*0 + B10 \\ &= B0 + B111^*0 + 1^*0 + B10 \end{aligned}$$

$$B = B(0 + 111^*0 + 10) + 1^*0$$

$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ R & P & Q \end{array}$

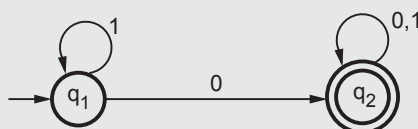
$$\therefore B = 1^*0(0 + 111^*0 + 10)^* \quad \dots (6)$$

Put equation (6) in equation (3),

$$C = 1^*0(0 + 111^*0 + 10)^*1 \text{ is R.E.}$$

Example 2.7.12 Obtain the regular expression that denotes the language accepted by the following DFA, using Arden's Theorem :

SPPU : Aug.-15, In Sem, Marks 4



Solution : Let us write the equations for each state

$$q_1 = q_11 + \varepsilon \quad \dots (1)$$

$$q_2 = q_10 + q_2(0 + 1) \quad \dots (2)$$

Let us solve equation (1) using Arden's theorem

$$q_1 = q_11 + \varepsilon$$

$\begin{array}{cccc} \downarrow & \downarrow & \downarrow & \downarrow \\ R & R & P & Q \end{array}$

$$\therefore q_1 = \varepsilon 1^* = 1^* \quad \because R = RP + Q \text{ gives } R = QP^*$$

$$\therefore q_1 = 1^* \quad \dots (3)$$

Putting equation (3) in (2) we get

$$q_2 = 1^*0 + q_2(0+1)$$

$\downarrow$
R

$\downarrow$
Q

$\downarrow$
R

$\downarrow$
P

$$\therefore q_2 = 1^*0(0+1)^* \quad \dots (4)$$

As q_2 is a final state equation for q_2 represents regular expression.

$$\therefore \text{r.e.} = 1^*0(0+1)^*$$

Example 2.7.13 Make use of Arden's theorem to determine the regular expression for the finite automata shown in Fig. 2.7.13.

SPPU : May-16 Marks 6

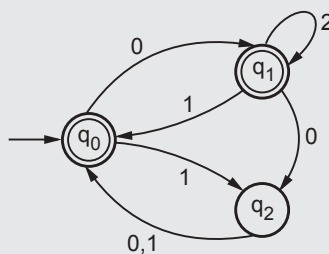


Fig. 2.7.13

$$\text{Solution : } q_0 = \varepsilon + q_11 + q_2(0+1) \quad \dots(1)$$

$$q_1 = q_00 + q_12 \quad \dots(2)$$

$$q_2 = q_01 + q_00 \quad \dots(3)$$

Now we will put equation (3) in (1)

$$q_0 = \varepsilon + q_11 + (q_01 + q_00)(0+1)$$

$$\text{i.e. } q_0 = \varepsilon + q_11 + q_010 + q_011 + q_100 + q_101$$

$$\text{i.e. } = \varepsilon + q_1(1+00+01) + q_01(0+1)$$

$$q_0 = \varepsilon + q_1(1+00+01) + q_01(0+1)$$

$\downarrow$
R

$\downarrow$
Q

$\downarrow$
R

$\downarrow$
P

$$\therefore q_0 = [\varepsilon + q_1(1+00+01)](1(0+1))^* \quad \dots (4)$$

Now solving equation (2) with the help of equation (4)

$$q_1 = q_00 + q_12$$

$$\begin{aligned} q_1 &= [\epsilon + q_1(1+00+01)](1(0+1))^* 0 + q_1 2 \\ &= (1(0+1))^* 0 + q_1(1+00+01)(1(0+1))^* 0 + q_1 2 \end{aligned}$$

$$q_1 = \underbrace{(1(0+1))^* 0}_{\downarrow R} + \underbrace{q_1}_{\downarrow Q} \underbrace{[(1+00+01)(1(0+1))^* 0 + 2]}_{\downarrow P}$$

$$\therefore q_1 = (1(0+1))^* 0 [(1+00+01)(1(0+1))^* 0 + 2]^*$$

From equation (4) the q_0 will be

$$\begin{aligned} q_0 &= (\epsilon + q_1 + (1+00+01))(1(0+1))^* \\ &= (1(0+1))^* + q_1(1+00+01)(1(0+1))^* \\ q_0 &= (1(0+1))^* + ((1(0+1))^* 0 (1+00+01)(1(0+1))^* 0 + 2) (1+00+01)(1(0+1))^* \end{aligned}$$

Now for final regular expression,

$$\text{R.E.} = q_0 + q_1$$

$$\begin{aligned} \therefore \text{R.E.} &= [(1+(0+1))^* + ((1(0+1))^* 0 (1+00+01) \\ &\quad ((1(0+1))^* 0 + 2)(1+00+01)(1(0+1))^* 0 + 2) (1+00+01)(1(0+1))^* 0 + 2] \end{aligned}$$

2.8 Pumping Lemma for Regular Languages

SPPU : Dec.-06, 08, 12, May-06, 07, 09, 12, 13, 14, 16, Aug.-17, Oct.-18, Marks 8

This is a basic and important theorem used for checking whether given string is accepted by regular expression or not. In short, this lemma tells us whether given language is regular or not.

One key theme is that any language for which it is possible to design the finite automata is definitely the regular language.

Theorem : Let L be a regular set. Then there is a constant n such that if z is any word in L and $|z| \geq n$ we can write $z = uv^i w$ such that $|uv| \leq n$, $|v| \geq 1$ for all $i \geq 0$, $uv^i w$ is in L . The n should not be greater than the number of states.

Proof : If the language L is regular it is accepted by a DFA. $M = (Q, \Sigma, \delta, q_0, F)$. With some particular number of states say, n . Consider the input can be $a_1, a_2, a_3, \dots, a_m$, $m \geq n$. The mapping function δ could be written as $\delta(q_0, q_1, q_2, q_3, \dots, q_i) = q_i$.

The transition diagram is as shown in Fig. 2.8.1.

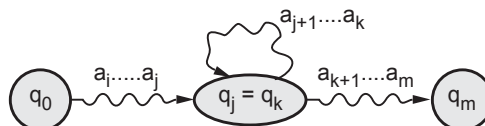


Fig. 2.8.1 Pumping lemma

If q_m is in F i.e. $q_1, q_2, q_3, \dots, q_m$ is in $L(M)$ then $a_1, a_2, \dots, a_j, a_{k+1}, a_{k+2}, \dots, a_m$ is also in $L(M)$. Since there is path from q_0 to q_m that goes through q_j but not around the loop labelled $a_{j+1} \dots a_k$. Thus

$$\begin{aligned}\delta(q_0, a_1, a_j, a_{k+1} \dots a_m) &= \delta(\delta(q_0, q_1, \dots, q_j), a_{k+1} \dots a_m) \\ &= \delta(q_j, q_{k+1} \dots q_m) \\ &= \delta(q_k, q_{k+1} \dots q_m) \\ &= q_m\end{aligned}$$

That is what we have proved i.e. given any long string can be accepted by FA, we should be able to find a substring near the beginning of the string that may be pumped i.e. repeated as many times as we like and resulting string may be accepted by FA.

The pumping lemma is used to check whether given language is **regular or not**.

2.8.1 Applications of Pumping Lemma

The pumping lemma is used to check whether given language is regular or not. We will discuss it with the help of some examples.

Example 2.8.1 Show that the set $L = \{b^{i^2} \mid i > 1\}$ is not regular.

SPPU : Dec.-12, Marks 6, Aug.-17, Oct.-18, In Sem, Marks 4

Solution : We have to prove that the language $L = b^{i^2}$ is not regular. This language is such that number of b's is always a perfect square.

For example, if we take $i = 1$

$$\begin{aligned}L &= b^{1^2} = b \quad \text{the length} = 1^2 \\ &= b^{2^2} = bbbb \quad \text{length} = 2^2\end{aligned}$$

and so on.

Now let us consider

$$L = b^{n^2} \text{ where } \text{length} = n^2$$

it is denoted by z .

$$|z| = n^2$$

By pumping lemma $z = u v w$

where $1 \leq |v| \leq n$

As $z = u v^i w$ where $i = 1$

Now we will pump v i.e. make $i = 2$.

As we made $i = 2$ we have added one n^2 .

$$1 \leq |v| \leq n.$$

$$n^2 + 1 \leq |uvw| \leq n + n^2$$

$$\text{i.e. } n^2 + 1 \leq |uvw| \leq n^2 + n + n + 1$$

$$\text{i.e. } n^2 + 1 \leq |uvw| \leq (n+1)^2$$

$$= n^2 \leq |uvw| \leq (n+1)^2$$

Thus the string lies between two consecutive perfect squares. But the string is not a perfect square. Hence we can say the given language is not regular.

For example,

$$L = b^{i^2}$$

Let $i = 2$

$$L = bbbb$$

$$L = uvw$$

Assume $uvw = bbbb$

Take $u = b$

$$v = bb$$

$$w = b$$

By pumping lemma, even if we pump v i.e. increase v then language should show the length as perfect square.

$$uvw$$

$$= uv.vw$$

$$= bbbbbb$$

$$= \text{length of } b \text{ is not a perfect square}$$

Thus the behaviour of the language is not regular, as after pumping something onto it does not show the same property (being square for this example.)

Example 2.8.2 Is the following language regular ? Justify your answer $L = \{0^{2n} \mid n \geq 1\}$.

Solution : This is a language in which length of string is always even.

i.e. $n = 1 ; \quad L = 00$

$$n = 2 \quad L = 00 \ 00 \quad \text{and so on.}$$

$$\begin{aligned}\text{Let } L &= uvw \\ L &= 0^{2n} \\ |z| &= 2n = uv^i w\end{aligned}$$

If we add $2n$ to this string length.

$$\begin{aligned}|z| &= 4n = uv.vw \\ &= \text{Even length of string.}\end{aligned}$$

Thus even after pumping $2n$ to the string we get the even length. So the language L is regular language.

Example 2.8.3 Prove $L = \{a^p \mid p \text{ is a prime}\}$ is not regular.

SPPU : Dec.-06, May-07, Marks 6, May-12, Marks 8

Solution : Let us assume L is a regular and p is a prime number.

$$\begin{aligned}L &= a^p \\ |z| &= uvw \quad i = 1\end{aligned}$$

$$\begin{aligned}\text{Now consider } L &= uv^i w \quad \text{where } i = 2 \\ &= uv.vw\end{aligned}$$

Adding 1 to p we get,

$$\begin{aligned}p &< |uvvw| \\ p &< p+1\end{aligned}$$

But $p+1$ is not a prime number. Hence what we have assumed becomes contradictory. Thus L behaves as it is not a regular language.

Example 2.8.4 Show that $L = \{0^n 1^{n+1} \mid n > 0\}$ is not regular.

Solution : Let us assume that L is a regular language.

$$\begin{aligned}|z| &= |uvw| \\ &= 0^n 1^{n+1}\end{aligned}$$

Length of string $|z| = n + n + 1 = 2n + 1$.

That means length is always odd.

By pumping lemma

$$= |uv.vw|$$

That is if we add $2n + 1$

$$2n + 1 < (2n + 1) + 2n + 1$$

$$2n + 1 < 4n + 2$$

But if $n = 1$ then we obtain $4n + 2 = 6$ which is no way odd. Hence the language becomes irregular.

Even if we add 1 to the length of $|z|$, then

$$|z| = 2n + 1 + 1 = 2n + 2$$

= Even length of the string.

So this is not a regular language.

The simple way to know whether given language is regular or not is that try to draw **finite automata** for it, if you can easily draw the FA for the given L then that language is surely the regular otherwise not.

Example 2.8.5 Show that set $L = \{0^i 1^i \mid i \geq 1\}$ is not regular.

Solution : Assume that $L = \{0^i 1^i \mid i \geq 1\}$ is regular.

Let, $w = 0^n 1^n$ such that $|w| = 2n$. By pumping lemma we can write

$$w = xyz \text{ such that } |xy| \leq n \text{ and } |y| \neq 0.$$

Now if $xy^i z \in L$ then the language L is said to be regular.

There are many cases – i) y has only 0's ii) y has only 1's iii) y has both 0's and 1's.

i) If y has only 0's then the string

$$w = 0^{n-k} 1^n = xz \text{ since } y = 0^k \text{ and } i = 0$$

Surely $n - k \neq n$. Hence $xz \notin L$.

Hence our assumption of being L regular is wrong.

ii) If y has only 1's then, for $i = 0$ and $y = 1^k$

$$\therefore w = xz = 0^n 1^{n-k}$$

As $n \neq n - k$, $xz = w \notin L$

Again L is not regular.

iii) If y has 0's and 1's then

$$w = 0^{n-k} 0^k 1^j 1^{n-j} = xy^i z$$

If $i = 2$ then

$$w = 0^{n-k} 0^{2k} 1^{2j} 1^{n-j} \notin L$$

Hence from all these 3 cases it is clear that language L is not regular.

Example 2.8.6 Use pumping lemma to show whether or not the following languages are regular :

$$i) A_1 = \{www \mid w \in (a, b)^*\} \quad ii) A_2 = \{a^{2n} \mid n \geq 0\}.$$

Solution : i) Let,

$A_1 = \{www \mid w \in (a, b)^*\}$ is assumed to be a regular language. Then by pumping lemma the string $w = xyz \in A_1$ such that

$$w = xy^i z$$

Here $|w| = |www| = 3n$ where $n > 0$

By pumping lemma if

$$\begin{aligned} w &= xy^i z \\ &= ww^i w \end{aligned}$$

$$|w| = > 3n$$

That means $w \notin L$. This shows that given language is not regular. Hence our assumption is wrong.

ii) $A_2 = \{a^{2n} \mid n \geq 0\}$ is a language in which length of string is always even.

i.e. If $n = 1$ then $A_2 = aa$

If $n = 2$ then $A_2 = aaaa$ and so on

If $w = xyz$ then

$$|w| = a^{2n} = xy^i z$$

Then $|w| > 2n$. That means we may get a string of odd length. Hence given language is not regular.

Example 2.8.7 Let L be any subset of 0^* . Prove that L^* is regular.

SPPU : May-07, 14, Marks 4

Solution : Let $L \subseteq 0^*$ i.e. L can be $\{\epsilon, 0, 00, 000 \dots\}$

Now we will apply pumping lemma to determine whether L is subset of 0^* .

Assume $w = xyz$ such that $|xy| \leq n$ $|y| \geq 0$

If we consider string $w = 0000$

We will map $w = xyz$ to 0000

By pumping lemma, if $w = xy^i z \in L$ the L is regular. If $i = 2$ then

$$\text{i.e. } w \in L$$

This proves that string w is regular. Hence if L is subset of 0^* then L^* is regular is proved.

Example 2.8.8 Using pumping lemma for regular sets prove that the language $L = \{a^m b^n \mid m > n \text{ is not regular}\}$

SPPU : Dec.-08, Marks 6

Solution : Let us assume

$$L = \{a^m b^n \mid m > n\} \text{ is regular.}$$

As $m > n$, if we assume $m = n + 1$ then

$$L = a^{n+1} b^n = a^n a b^n$$

By pumping lemma $L \in W$ such that

$$W = xyz$$

If we map $L = a^n a b^n$ to W then

$$W = a^n a b^n. \text{ Assume } x = a^n, y = ab, z = b^{n-1}$$

If $W = xy^i z$ with $i = 2$ then

$$W = a^n a b a b b^{n-1} \notin L.$$

$$W = a^n b^n \notin L$$

This proves that our assumption of L being regular is wrong. Given L is not regular.

Example 2.8.9 Prove that the following is not regular languages the set of strings of the form $0^i 1^j$ such that the greatest common divisor of i and j is 1.

Solution : Let $L = 0^i 1^j$ such that $\gcd(i, j) = 1$.

We assume that this is a regular language. Now consider three cases -

Case 1 : Let

$$z = \overbrace{0 \dots 0}^i \overbrace{1 \dots 1}^j \text{ be a string } \in L.$$

According to pumping lemma if $z = uvw$ is a string $\in L$, then $z = uv^i w \in L$. That means even if we pump some substring the string $z \in L$.

We map $z = uvw$ as

$$z = \underbrace{0 \dots 0}_{u} \underbrace{0 \dots 0}_{v} \underbrace{1 \dots 1}_{w}$$

This is a case in which v consists of substring $0^n 1^n$.

By pumping lemma $z = uv^i w$ if $i = 2$, $z = uvvw$

Then we get

$$z = 0^{i-m} (01)^{m-1} 1^{j-m} \notin L = \{0^i 1^j\}$$

Case 2 : Let,

$$z = \underbrace{0 \dots 0}_u \underbrace{1 \dots 1}_w \in L$$

By pumping lemma $z = uv^i w$. If $i = 2$ then,

$$z = uvvw$$

i.e. $z = 0^{i+m} 1^j$

But $\gcd(i+m, j) \neq 1$.

Hence $z \notin [L = \{0^i 1^j\}]$

Case 3 : Let,

$$z = \underbrace{0 \dots 0}_u \underbrace{1 \dots 1}_{vw}$$

By pumping lemma, $z = uv^i w$. If $i = 2$ then,

$$z = uvvw$$

i.e. $z = 0^i 1^{j+m}$

But $\gcd(i, j+m) \neq 1$.

Hence $z \notin [L = \{0^i 1^j\}]$

From above 3 cases we can prove that $L = \{0^i 1^j \mid \gcd(i, j) = 1\}$ is not a regular language.

Example 2.8.10 Prove or disprove that the language L given by $L = \{a^m b^n \mid m \neq n, m \text{ and } n \text{ are positive integers}\}$ is regular.

SPPU : Aug.-17, In Sem, Marks 4

Solution : Let $L = \{a^m b^n \mid m \neq n\}$ be a language. Assume that L is regular language. Now consider following case -

Case 1 : Assume $z = aaabbbb \in L$

By pumping lemma if $z = uvw$ then if we pump some strings to make $z = uv^i w$ then if $z \in L$ then such a language is called regular.

Let,

$$z = \begin{array}{ccccccc} a & a & a & b & b & b & b \\ \downarrow & & \downarrow & & \downarrow & & \\ u & & v & & w & & \end{array}$$

if $z = uv^i w$ and if $i = 2$ then $z = uvvw$

$$\therefore \quad \begin{array}{ccccccc} z & = & a & (a & a & b & b) & (a & a & b & b) & b & b \\ & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow \\ & & u & & v & & v & & & & w & & \end{array}$$

$$z = a^3 b^2 a^2 b^4 \notin a^m b^n \notin L$$

Case 2 : Assume $L \in z$ such that

$$\begin{array}{ccccccc} z & = & a & a & a & a & b & b & b & b & b & b \\ & & \downarrow & \downarrow & \downarrow & & \downarrow & & & & & \\ & & u & & v & & & & w & & & \end{array}$$

By pumping lemma, $z = uv^i w \in L$ for a regular language. If $i = 2$ then,

$$z = uvvw$$

$$\therefore \quad \begin{array}{ccccccc} z & = & a & a & a & a & a & a & b & b & b & b & b \\ & & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & & \downarrow & & & & \\ & & u & & v & & v & & & & w & & \end{array}$$

$$z = a^6 b^6 \in a^m b^n \text{ because } m = n.$$

From these case we get $z \notin L$. Thus our assumption of L being regular is wrong. Hence given language $\{ L = a^m b^n \mid m \neq n \}$ is not a regular.

Example 2.8.11 Show that $L = \{ ww \mid w \in \{a, b\}^* \}$ is not regular.

SPPU : Dec.-12, Marks 6

Solution : Assume that the language L is regular. This also means that there exists some DFA with n states.

If we consider $w = a^n b$ i.e. string of $\{a, b\}$

then $ww = a^n b a^n b$ is in L

i.e. $|ww| = 2n+2 > n$

by pumping lemma, we can write

$$ww = xyz \text{ with } |y| = 0 \text{ } |xy| \leq n$$

iii) For checking regularity we need to find whether $xy^i z \in L$ or not.

Case 1 : $w = xy^i z$

we assume $i = 0$. This is a case with y having no b 's.

Then $w = xy$ i.e. $y = a^k$

Y containing no b 's at all

$$w = a^m b a^n b$$

where $m = n - k$

Then we can not write xz in the form of ww . Because w belongs to set $\{a, b\}^*$.

Case 2 : This is a case with y having only one b .

Then assume

$$w = xy^i z$$

These $w = xz$ with only one b .

But then $xz \notin L$ as in language L there should be even number of a 's and even number of b 's. [we have assumed $|ww| = 2n + 2$].

Thus from case 1 and case 2 we get the contradictions. So L is not regular.

Example 2.8.12 Define Pumping Lemma and apply it to prove the following :

$L = \{0^m 1^n 0^{m+n} \mid m \geq 1 \text{ and } n \geq 1\}$ is not regular. **SPPU : May-09, 16, Marks 6**

Solution : Assume that L is regular by pumping lemma, we can write $w = xyz$ with $|y| = 0 < |xy| \leq n$

For checking the regularity we need to find whether $xy^i z \in L$ or not.

$$\begin{aligned} w &= xyz \\ &= 0^m 1^n 0^{m+n} \end{aligned}$$

Case 1 : If we assume y has only 1's i.e.

$$\begin{array}{ccccc} w &= & 0^m & 1^n & 0^{m+n} \\ & & \downarrow & \downarrow & \downarrow \\ & & x & y & z \end{array} \quad \text{then with } i = 2,$$

$$w = xy^i z \text{ becomes } xyyz$$

$$\text{i.e. } = 0^m 1^n 1^n 0^{m+n}$$

$$w = 0^m 1^{2n} 0^{m+n} \notin L$$

Case 2 : If we assume y is set of $x y 0$ i.e.

$$\begin{array}{ccccc} w &= & 0^{m-k} & 0^k 1^{n-p} & 1^p 0^{m+n} \\ & & \downarrow & \downarrow & \downarrow \\ & & x & y & z \end{array}$$

then with $i = 2$ $w = xyyz$

$$\text{i.e. } w = 0^{m-k} (0^k 1^{n-p}) (0^k 1^{n-p}) 1^p 0^{m+n}$$

$$w = 0^{m-k} 0^{2k} 1^{2n-2p} 1^p 0^{m+n}$$

$$= 0^{m-k} 1^{2n-p} 0^{m+n} \notin L$$

From above two cases by applying pumping lemma i.e. by using

$$w = xy^i z \notin L$$

Hence our assumption of L being regular is wrong.

∴ Given L is not regular.

Example 2.8.13 Find whether the language is regular or not

$$L = \{WW^R \in \{a, b\}^*\}$$

Solution : Consider, $L = \{w \in \{a, b\}^* \mid w = w^R\}$

Consider $w = abab$

$$w^R = baba$$

Hence $L = ww^R$

The string z can be denoted by

$$z = uvw. \text{ where } |z| \geq n \text{ and } |uv| \leq n \text{ } |v| \geq 1.$$

$$\text{We assume } z = \underbrace{ab}_n \underbrace{ab}_n \underbrace{ba}_n \underbrace{ba}_n$$

We can split $z = uvw$ as

$$z = \underbrace{a}_u \underbrace{b}_1 \underbrace{ab ba ba}_w$$

$$|u| = n - 1$$

$$|v| = 1$$

$$\begin{aligned} \text{i.e. } |uv| &= |u| + |v| \\ &= n - 1 + 1 \\ &= n \end{aligned}$$

Now according to pumping lemma,

when $z = uv^i w \in L$ then language is said to be regular. We will assume that $L = ww^R$ is regular. And we will find $uv^i w$.

$$\therefore z = uv^i w = ababbaba$$

Assume $i = 0$ i.e. there will be

$$z = uv^0 w = uw$$

Then the string becomes

$$z = aabbaba$$

$$\neq ww^R$$

$$\text{i.e. } z = aabbaba \notin L \quad \dots(1)$$

Now consider $i = 2$ then

$$\begin{aligned} z &= uv^2w \\ &= abbabbaba \\ &\neq ww^R \end{aligned}$$

i.e. $z = abbabbaba \notin L \quad \dots(2)$

From equations (1) and (2) we can state that

$$z = uv^i w \notin L$$

Hence our assumption that $L = ww^R$ being regular is wrong. Hence we can prove that

$$z = ww^R \text{ is not regular.}$$

Review Questions

1. Explain the property pumping lemma of regular set. Where do we apply this property ? Give the suitable example. **SPPU : May-06, Marks 6**
2. State pumping lemma for regular sets. **SPPU : May-13, Marks 2**

2.9 Closure Properties of Regular Languages

SPPU : May-18, Oct 18, Marks 6

If certain languages are regular and language L is formed from them by certain operations (such as union or concatenation) then L is also regular. These properties are called closure properties of regular languages. Such languages represent the class of regular languages which is closed under the certain specific operations.

The closure properties express the idea that when one or many languages are regular then certain related languages are also regular. The closure properties of regular languages are as given below.

1. The union of two regular languages is regular.
2. The intersection of two regular languages is regular.
3. The complement of a regular languages is regular.
4. The difference of two regular languages is regular.
5. The reversal of a regular languages is regular.
6. The closure operation on a regular language is regular.
7. The concatenation of regular language is regular.
8. A homomorphism of regular languages is regular.
9. The inverse homomorphism of regular language is regular.

Theorem 1 : If L_1 and L_2 are two languages then $L_1 \cup L_2$ is regular.

Proof : If L_1 and L_2 are regular then they have regular expression $L_1 = L(R_1)$ and $L_2 = L(R_2)$. Then $L_1 \cup L_2 = L(R_1 + R_2)$ thus we get $L_1 \cup L_2$ as regular language. (any language given by some regular expression is regular).

Theorem 2 : The complement of regular language is regular.

Proof : Consider L_1 be regular language which is accepted by a DFA $M = (Q, \Sigma, \delta, q_0, F)$. The complement of regular language is $\bar{L}_1$ which is accepted by $M' = (Q, \Sigma, \delta, q_0, Q - F)$. That means M is a DFA with final states $\in F$ and M' is a DFA in which all the non-final states of M become final. In other words, we can say that the strings that are accepted by M are rejected by M' similarly, the strings rejected by M are accepted by M' .

Thus as $\bar{L}_1$ is accepted by DFA M' , it is regular.

Theorem 3 : If L_1 and L_2 are two regular languages then $L_1 \cap L_2$ is regular.

Proof : Consider that languages L_1 is regular. That means there exists some DFA M_1 that accepts L_1 . We can write $M = (Q_1, \Sigma, \delta_1, q_1, F_1)$. Similarly being L_2 regular there is another DFA $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

Let, L be the language obtained from $L_1 \cap L_2$. We can simulate $M = (Q, \Sigma, \delta, q, F)$.

where $Q = Q_1 \cap Q_2$

$\delta = \delta_1 \cap \delta_2$ a mapping function derived from both the DFAS.

$q \in Q$ which is initial state of machine M .

$F = F_1 \cap F_2$, the set of final states, which is common for M_1 and M_2 both.

It is clear that there exists some DFA which accepts $L_1 \cap L_2$ i.e. L . Hence L is a regular language. This proves that if L_1 and L_2 are two regular languages then $L_1 \cap L_2$ is regular. In other words the regular language is closed under intersection.

Theorem 4 : If L_1 and L_2 are two regular languages then $L_1 - L_2$ is regular.

Proof : The $L_1 - L_2$ can also be denoted as $L_1 \cup \bar{L}_2$

Consider L_1 be regular language which is accepted by DFA $M = (Q, \Sigma, \delta, q_0, F)$. The complement of regular language L_1 is $\bar{L}_1$ which is accepted by $M' = (Q, \Sigma, \delta, q_0, Q - F)$. That means M is a DFA with final state's set F and M' is a DFA in which all the non final states of M become final states and all the final states of M become non-final states. Thus L_1 and $\bar{L}_2$ are two regular languages. That also means : these languages are accepted by regular expressions. If $L_1 = L(R_1)$ and $\bar{L}_2 = L(R'_2)$. Then $L_1 \cup \bar{L}_2 = L(R_1 + R'_2)$. This ultimately shows that $L_1 \cup \bar{L}_2$ is regular. In other words $L_1 - L_2$ is regular. Thus regular languages are closed under difference.

Theorem 5 : The reversal of a regular languages is regular.

Proof : Reversal of a string means obtaining a string which is written from backward that is w^R is denoted as reversal of string w . That means $L(w^R) = (L(w))^R$. This proof can be done with basis of induction.

Basis : If $w = \epsilon$ or ϕ then w^R is also ϵ or ϕ

i.e. $(\epsilon)^R = \epsilon$ and $(\phi)^R = \phi$

Hence $L(w^R)$ is also regular.

Induction :

Case 1 : If $w = w_1 + w_2$ then

$$w = (w_1)^R + (w_2)^R$$

As the regular language is closed under union. Then w is also regular.

Case 2 : If $w = w_1 w_2$

Consider $w_1 = (ab, bb)$

and $w_2 = (bbba, aa)$

The $w_1^R = (ba, aa)$

$w_2^R = (aaab, bb)$ then

$$w = w_1^R w_2^R$$

$(ba, aa, aaab, bb)$ is also regular. Thus given language is regular one.

Theorem 6 : The closure operation on a regular language is regular.

Proof : If language L_1 is regular then it can be expressed as $L_1 = L(R_1^*)$. Thus for a closure operation a language can be expressed as a language of regular expressions. Hence L_1 is said to be a regular language.

Theorem 7 : If L_1 and L_2 are two languages then $L_1 \cdot L_2$ is regular. In other words regular languages are closed under concatenation.

Proof : If L_1 and L_2 are regular then they can be expressed as $L_1 = L(R_1)$ and $L_2 = L(R_2)$. Then $L_1 \cdot L_2 = L(R_1 \cdot R_2)$ thus we get a regular language. Hence it is proved that regular languages are closed under concatenation.

Theorem 8 : A homomorphism of regular languages is regular.

Proof : The term homomorphism means substitution of string by some other symbols.

For instance the string "aabb" can be written as 0011 under homomorphism. Clearly here, a is replaced by 0 and b is replaced by 1. Let Σ is the set of input alphabets and Γ be the set of substitution symbols then $\Sigma^* \rightarrow \Gamma^*$ is homomorphism. The definition of homomorphism can be extended as

$$\begin{aligned}\text{Let,} \quad w &= a_1 a_2 \dots a_n \\ h(w) &= h(a_1) h(a_2) \dots h(a_n)\end{aligned}$$

If L is a language that belongs to set Σ , then the **homomorphic image** of L can be defined as :

$$h(L) = \{ h(w) : w \in L \}$$

To prove that if L is regular $h(L)$ is also regular consider following example -

$$\text{Let,} \quad \Sigma = \{a, b\} \text{ and } w = abab$$

$$\text{Let} \quad h(a) = 00$$

$$\text{and} \quad h(b) = 11$$

$$\begin{aligned}\text{Then we can write } h(w) &= h(a) h(b) h(a) h(b) \\ &= 00110011\end{aligned}$$

The homomorphism to language is applied by applying homomorphism on each string of language.

$$\therefore \quad \text{If } L = ab^*b \text{ then,}$$

$$L = \{ab, abb, abbb, abbbb, \dots\}$$

$$\text{Now} \quad h(L) = \{0011, 001111, 00111111, 0011111111, \dots\}$$

$\therefore \quad h(L) = 00(11)^+$ As it can be represented by a regular expression, it is a regular language. Hence it is proved that if L is regular then $h(L)$ is also regular. In other words, family of regular languages is closed under homomorphism.

Theorem 9 : The inverse homomorphism of regular language is regular.

Proof : Let $\Sigma^* \rightarrow \Gamma^*$ is homomorphism.

The Σ is the input set and Γ be the substitution symbols used by homomorphic function.

Let, L be the regular language where $L \in \Sigma$, then $h(L)$ be homomorphic language.

The inverse homomorphic language can be represented as $h^{-1}(L)$

Let,

$$h^{-1}(L) = \{w \mid w \in L\}$$

If L is regular then $h(L)$ is also regular because regular language is closed under homomorphism. That if there exist a FA $M = (Q, \epsilon, \delta, q_0, F)$ which accepts L then $h(L)$ must also be accepted by FA M . For complement of L i.e. language L' the inverse homomorphic language is $h^{-1}(L)$. Let M' be the FA in which all the final states of M

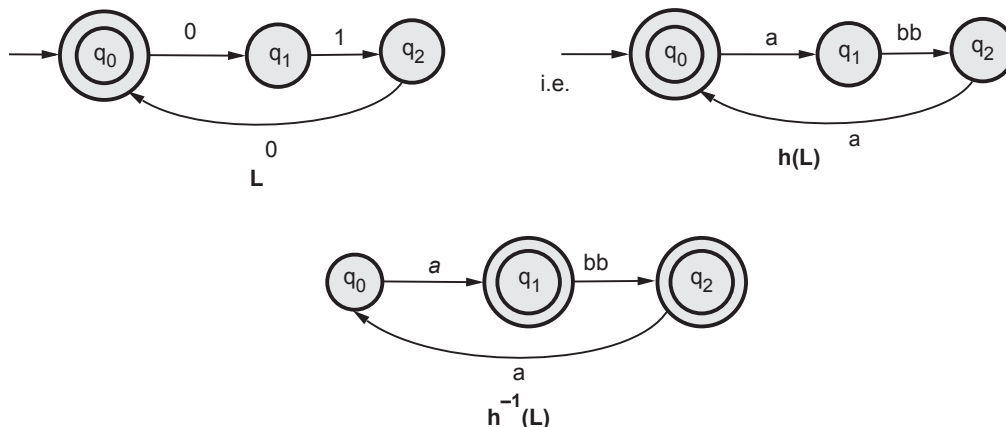
become non-final states and all the non-final states of M become the final states. Clearly the language L' can be accepted by M' . Hence $h^{-1}(L)$ must also be accepted by FA M' .

For example - Let $L = (010)^*$ be a regular language. And $L = \{010, 010010, \dots\}$

Let $h(0) = a$ and $h(1) = bb$ be homomorphic function. Then

$$h(L) = (abba)^*$$

This L can be represented by following DFA.



Thus there exists a FA which accepts $h^{-1}(L)$. This shows that, inverse homomorphism of regular language is regular.

Review Questions

1. With respect to properties of regular languages explain what is pumping lemma and closure properties of regular languages.

SPPU : May-18, End Sem, Marks 6

2. Prove that "Regular language is closed under complementation"

SPPU : Oct 18, In Sem, Marks 3

2.10 Decision Properties of Regular Languages

Following is the set of questions for determining the decision properties of regular languages.

1. Is a particular string w belongs to some language L ? [**Membership Property**]

Let L be a language which can be represented by DFA. If a string w is chosen for simulating this DFA processing and then if w reaches to an accept state starting from the start state then it is said that.

2. Is the given language empty ? **[Emptiness property]**

The regular language is modelled by a finite automata to decide whether it is empty or not. If there exists some path from start state to accept state for deriving such language then the given language is non empty. But if accepting states are separated from start state or in other words if there is no path reaching to an accept state from a start state while deriving a given language then such a language is called empty language.

3. Do the two descriptions of a language represent the same language ? or whether two languages are equivalent ? **[Equivalence property]**

Any regular language can be represented by NFA, NFA with DFA or by regular expression. These representations can be interconverted. This property is called equivalence of corresponding two models. For instance : NFA to DFA, DFA to r.e. and so on. If two different descriptions can be modelled by two equivalent but different models, the one can decide whether they are describing the same language or not.

2.11 Limitation of FA

SPPU : Aug.-15, Marks 2

The main limitation of FA is that it can not remember arbitrarily long input sequence because it does not contain any memory. Hence it can not solve following types of problems -

1. Checking well formedness of parenthesis
2. Checking palindrome condition of given language.

Thus FA cannot accept non regular and context free languages. It can accept only regular languages.

Review Question

1. What are the limitations of finite automata ? Justify with suitable examples

SPPU : Aug.-15, In Sem, Marks 2

2.12 Myhill - Nerode Theorem

The Myhill - Nerode theorem was proposed by **John Myhill and Anil Nerode** who proved it at University of Chicago. This theorem is used to prove that the given language is not regular. The theorem can be stated as follows -

Theorem - Given a language L , define a relation R_L on strings by rule $xR_L y$ if there is no distinguishing extension z with the property that exactly one of the strings xz and yz is in L .

The Myhill - Nerode theorem is useful for indistinguishability of strings i.e. Strings x and y in Σ^* are indistinguishable with respect to a language L if and only if for any string z in Σ^* , either xz and yz are both in L or they are both not in L .

For example, a and aa are indistinguishable with respect to the language a^n over alphabet $\{a\}$, where n is a positive integer, because aa^k and aaa^k are in the language a^n for any positive integer k . However, with respect to the language $a^n b^n$, a and aa are not indistinguishable (hence distinguishable), because ab is in the language $a^n b^n$ while aab is not in the language. Using this concept of indistinguishability, the theorem by **Myhill and Nerode** gives a criterion for non-regularity of a language. Hence we can state the theorem in other words as,

"A language L over alphabet Σ is nonregular if and only if there is an infinite subset of Σ^* , whose strings are pairwise distinguishable with respect to L ".

Example 2.12.1 Prove that $L = \{a^n b^n \mid n > 0\}$ is non regular using Myhill - Nerode theorem.

Solution : We will assume a set of strings $S = \{a^n \mid n > 0\}$. The set S is over alphabet $\{a, b\}$ and it is infinite. We have to show that its strings are pairwise distinguishable with respect to language L .

Assume that a^i and a^j be arbitrary two different members of the set S , where i and j are positive integers and $i \neq j$.

Let b^j as a string to be appended to a^i and a^j . It then becomes $a^i b^j$ and $a^j b^j$. From these two string $a^i b^j$ is not in L while $a^j b^j$ is in L . Hence we can say that a^i and a^j are arbitrary strings of S and are distinguishable with respect to L . Thus S satisfies the conditions of Myhill-Nerode theorem. This proves that L is non-regular.

Example 2.12.2 $L = \{ww \mid w \in \{a, b\}^*\}$ is non-regular.

Solution : We will assume a set of strings $S = \{a^n \mid n > 0\}$. The set S is over alphabet $\{a, b\}$ and it is infinite. We have to show that its strings are pairwise distinguishable with respect to language L .

Assume that a^i and a^j be arbitrary two different members of the set S , where i and j are positive integers and $i \neq j$. Select $ba^i b$ as a string to be appended to a^i and a^j . Then $a^i b a^i b$ is in L while $a^j b a^i b$ is not in L . Here a^i and a^j are arbitrary strings of S and are distinguishable with respect to L . Thus S satisfies the conditions of Myhill-Nerode theorem. Hence L is non-regular.

2.13 Case Study**SPPU : Dec.-05, 06, 10, 13, 14, May-13, 14, Oct.-19, Marks 6****1. Regular expressions in UNIX**

There are various UNIX notations used for regular expressions. These notations have many additional capabilities and features. These notations have ability to name and refer previous strings that have a matched pattern. This ability helps in recognizing nonregular languages. UNIX regular expressions allow to write **character classes**. There are some rules for these character classes which are as given below -

- 1) The symbol `.` stands for any single character.
- 2) The sequence `[1, 2, 3, ... 10]` means `1 + 2 + 3 + .. + 10`.
- 3) The range of characters can be specified using square brackets.

For example : `[a-z]` denotes set of lower case letters.

- 4) For matching with some special character a backslash is used.

For example : `\-` means match with character `-`.

- 5) Special notations can be used to represent some common class of characters.

For example : `[:digit:]` represents the class `[0-9]`. The `[:alpha:]` is same as `[A-Za-Z]`.

- 6) The operator `|` is used to denote union.
- 7) `?` is used to denote preceding zero or single character.

For example : `- ? [0-9]` means the number can be positive or negative number.

- 8) `+` is used to denote **one or more** occurrences.
- 9) `*` is used to denote **zero or more** occurrences.

- 10) `{n}` means **n copies of**.

For example : `a {3}` means `aaa`.

2. Lexical analyzers

Compiler uses this program of lexical analyzer in the process of compilation. The task of lexical analyzer is to scan the input program and separate out the 'tokens'.

For example : Identifier is a category of token in the source language and it can be identified by regular expression as

(letter) (letter + digit)*

If anything in the source language matches with this regular expression then it is recognized as identifier. The letter is nothing but a set `{A, B, ... Z, a, b, ... z}` `*` and digit is `{0, 1, ... 9}` `*`. The regular expression is effective way for identifying token from a language.

We can represent the above regular expression using a finite automata as follows -

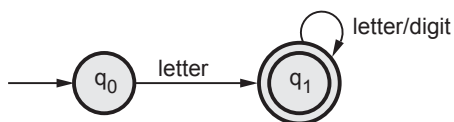


Fig. 2.13.1

3. Finding patterns in text

Regular expressions are useful notations used for finding the patterns from given text. A large class of pattern can be identified by regular expression.

For example : The strings ending with digits is a pattern in the given text which can be recognized by following regular expression -

`[a-zA-Z] + [0-9] +`

Thus use of regular expressions for identifying patterns from given text makes the job of compiler more simplified.

In web applications also the pattern matching is done using regular expressions.

4. GREP utility in Unix

In Unix there is a GREP utility for searching the desired pattern in the file. For searching the pattern from the Grep makes use of regular expressions. When particular string is present in the file, then using grep we get the line containing that string from the file.

For example : If we type the grep command on Unix prompt then we will get the output as follows `$ grep hello test.txt`.

hello friends

`$ grep b.g test.txt`.

big brother ← these are the lines

bug ← present in **test.txt**.

bag of books ← file containing

bigger ← `b[any single character] g`

Here dot stands for matching with one character.

Review Questions

1. Explain the following applications of regular expression.
 - i) GREP utilities in unix.
 - ii) Finding pattern in text.

SPPU : Dec.-05, 06, May-14, Marks 6

2. Explain the use of regular expressions in unix with any one example.

SPPU : Dec.-10, Marks 4

3. Explain the applications of regular expressions in lexical analysis phase of compiler.

SPPU : Dec.-13, May-13, Marks 6

4. State applications of Regular Expression

SPPU : Dec.-14 ,End Sem, Marks 4

5. Explain the application of regular expression in GREP utilities in Unix

SPPU : Oct.-19, In Sem, Marks 5

2.14 Fill in the Blanks

Q.1 The zero or more occurrences of a letter is denoted using ____ operator.

Q.2 The one or more occurrences of a letter is denoted using ____ operator.

Q.3 The set of strings that end in 0 is denoted as ____.

Q.4 The theorem for checking non regularity of a language is ____.

Q.5 If P, Q, R are three regular expressions and if $R = Q + RP$ then it has a unique solution given by ____.

Q.6 The RE in which any number of 0's is followed by any number of 1's followed by any number 2's is ____.

Q.7 In regular expression for adjoining two symbols we use ____ operator.

Q.8 In regular expression the null or empty string is denoted by ____.

Q.9 Concatenation of R with $\emptyset$ gives ____.

Q.10 $\emptyset^* =$ ____.

Q.11 If A and B are regular languages then A^* is ____.

2.15 Multiple Choice Questions

Q.1 Regular expressions are used to represent which language ____.

☐ a recursive language

☐ b context free language

☐ c regular language

☐ d all of these

Q.2 The set of all strings over $\Sigma = \{a,b\}$ in which all strings of a's and b's ending in bb is ____.

☐ a ab

☐ b a^*bbb

☐ c $(a+b)^* bb$

☐ d all of these

Q.3 Which of the following is not a prefix of abc ?

☐ a ϵ

☐ b a

☐ c ab

☐ d bc

Q.4 What is $R+R$ _____.

☐ a R

☐ b 0

☐ c $R+$

☐ d None

Q.5 Any transition graph has an equivalent _____.

☐ a RE

☐ b DFA

☐ c NFA

☐ d DFA, NFA, RE

Q.6 Which of the following is not a regular expression ?

☐ a $(a+b)^*abb$

☐ b $abba^*$

☐ c $0 + 1$

☐ d $(a+b)^*-(aa)$

Q.7 Regular expressions are _____.

☐ a type 0 language

☐ b type 1 language

☐ c type 2 language

☐ d type 3 language

Q.8 Regular expressions are closed under _____.

☐ a union

☐ b intersection

☐ c kleen star

☐ d all of these

Q.9 Regular expression $(a/b)(a/b)$ denotes the set of _____.

☐ a $\{ab,ab\}$

☐ b $\{aa,bb,ab,ba\}$

☐ c $\{aa,bb\}$

☐ d $\{a,b\}$

Q.10 Which of the following identity is wrong ?

☐ a $R + R = R$

☐ b $(R^*)^* = R^*$

☐ c $\epsilon R = R\epsilon = R$

☐ d $\emptyset R = R\emptyset = RR^*$

Q.11 Concatenation operation refers to which of the following set operations :

☐ a Union

☐ b Dot

☐ c Kleene

☐ d a and c

Answer Keys for Fill in the Blanks :

Q.1	*	Q.2	+
Q.3	$(0 + 1)^*0$	Q.4	pumping lemma
Q.5	QP^*	Q.6	$0^*1^*2^*$
Q.7	dot or	Q.8	*
Q.9	$\emptyset$	Q.10	$\emptyset$
Q.11	regular		

Answer Keys for Multiple Choice Questions :

Q.1	a	Q.2	c	Q.3	d	Q.4	a
Q.5	d	Q.6	d	Q.7	a	Q.8	d
Q.9	b	Q.10	d	Q.11	b		

